# AUTOFORGE: A CHECKPOINTED, HUMAN–IN–THE–LOOP MULTI-AGENT SDLC PIPELINE

**INDIVIDUAL THESIS**

SANSALA T.G.B.D

IT22174826

**Individual component:** Architecture Planning and UI/UX Design Generation

**Group ID:** R26-SE-033

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Software Engineering

Department of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

# DECLARATION AND SUPERVISOR CERTIFICATION

**Candidate declaration.** I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

**Academic integrity and AI disclosure.** I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Sansala T.G.B.D

Student ID: IT22174826

Signature:  Date: 05/10/2026

**Supervisor certification.** The above candidate carried out this research for the bachelor's degree thesis under my supervision.

| Role and name | Signature | Date |
|---|---|---|
| Supervisor: **Prof. Nuwan Kodagoda** | | |
| Co-supervisor: **Mr. Eishan Weerasinghe** |  | 10/10/2026 |

# ABSTRACT

In an LLM-based development pipeline, the design stages sit between an approved specification and the code. If the architecture plan misses a requirement, or a screen design shows something nobody asked for, the Coder Agent builds it anyway. This thesis covers the Architecture and UI/UX agents of AutoForge, a seven-agent, human-gated pipeline for e-commerce web development. The Architecture Agent produces a fifteen-section plan through a four-rung ladder: a read-only agentic exploration of earlier approved plans, a single-shot call, a JSON repair call and a deterministic fallback built from the SRS. A validator checks that every functional requirement, acceptance criterion, validation rule and non-functional requirement is traced. Use case, sequence and class diagrams come from narrow specifications that deterministic modelers normalize to UML rules before validators check them and PlantUML renders them. The UI/UX Agent plans pages and components, checks their coverage of the SRS, generates each component as an HTML and Tailwind fragment behind a placeholder check, and assembles self-contained previews. Shared design tokens change only after approval. I evaluated the work with 361 automated tests, the team's three-feature case study and stored artefacts. In the case study, the architecture and UI/UX stages both needed revisions that reviewers caught before code existed. The artefacts also show the limits of deterministic checking. A fallback plan passed full traceability while its data entity held fields such as "string" and "optional", because the Requirement Agent wrote "name: type" while the parser expected "name (details)". The UI/UX previews were complete and realistic, yet one enriched requirement had no page, and the two agents chose different routes. Traceability checks prove that something was planned, not that it was planned well, so human review of the design stages remains necessary.

**Keywords:** software architecture generation, UML, UI generation, human-in-the-loop, multi-agent systems

# ACKNOWLEDGEMENTS

I would like to thank my supervisor, Prof.Nuwan Kodagoda, and my co-supervisor, Mr.Eishan Weerasinghe, for their guidance throughout this project. Their questions about how we would know whether a generated diagram was actually correct pushed me to look past whether the output was well formed.

I am grateful to my teammates, Santhuka D.N.M.D., Dissanayake T.C. and Oberathna R.D.T.D. The specifications from the first stages were the input to my agents, and the Coder Agent was the first real consumer of the plans and page designs they produced. Many of the fixes described in this thesis began as a problem one of them found at the boundary between our components.

I also thank the academic staff of the Department of Computing at the Sri Lanka Institute of Information Technology, and the panel members at the progress presentations, for their feedback. Finally, I thank my family and friends for their patience and support during this project.

# TABLE OF CONTENTS

1 INTRODUCTION ..... 1
1.1 Overall project context ..... 1
1.2 Individual component context ..... 2
1.3 Problem statement ..... 3
1.4 Research gap ..... 3
1.5 Aim and objectives ..... 4
1.6 Research questions or hypotheses ..... 4
1.7 Scope limitations and assumptions ..... 5
1.8 Individual contribution ..... 5
1.9 Thesis structure ..... 6
2 LITERATURE REVIEW AND RELATED WORK ..... 7
2.1 Review method and scope ..... 7
2.2 Domain and theoretical foundations ..... 7
2.2.1 Architecture plans and design views ..... 7
2.2.2 UML diagrams ..... 8
2.2.3 Traceability into design ..... 8
2.2.4 Human oversight of agent plans ..... 8
2.2.5 Design science ..... 8
2.3 Related research ..... 9
2.4 Existing systems products and techniques ..... 10
2.5 Gap synthesis and conceptual framework ..... 11
3 RESEARCH METHODOLOGY ..... 12
3.1 Research design ..... 12
3.2 Data participants or study materials ..... 12

3.3 Experimental or study procedure ..... 13
3.4 Measures and evaluation metrics ..... 14
3.5 Data analysis ..... 14
3.6 Ethics privacy security and responsible AI ..... 14
3.7 Reproducibility and research artefacts ..... 15
4 PRODUCT AND SOLUTION DESIGN ..... 16
4.1 Requirements ..... 16
4.2 Component architecture within the complete solution ..... 17
4.3 Design alternatives and decisions ..... 18
4.4 Detailed design ..... 19
4.4.1 Architecture plan structure ..... 19
4.4.2 Generation ladder and coverage validation ..... 20
4.4.3 UML diagram generation ..... 21
4.4.4 Revision ..... 23
4.4.5 UI/UX page and component design ..... 23
4.4.6 Approval, cascades and the design system ..... 25
4.5 Standards and professional practice ..... 25
5 IMPLEMENTATION INTEGRATION AND TESTING ..... 27
5.1 Development environment and technology stack ..... 27
5.2 Component implementation ..... 27
5.2.1 Architecture Agent ..... 28
5.2.2 Diagram pipeline ..... 29
5.2.3 UI/UX Agent ..... 30
5.2.4 Interface support ..... 30
5.3 Integration with the other components ..... 30
5.4 Verification and software testing ..... 31

5.5 Deployment and operation ..... 32
6 RESULTS AND VALIDATION ..... 33
6.1 Research results ..... 33
6.2 Product and technical results ..... 34
6.2.1 Traceability of architecture plans (RQ1) ..... 34
6.2.2 Quality of the generated UML diagrams (RQ2) ..... 35
6.2.3 Completeness of the page designs (RQ3) ..... 38
6.2.4 Human review at the design stages (RQ4) ..... 40
6.3 User or stakeholder validation ..... 40
6.4 Integration results ..... 41
6.5 Summary of findings ..... 42
7 DISCUSSIONS ..... 43
7.1 Interpretation of findings ..... 43
7.2 Comparison with literature baselines and existing solutions ..... 44
7.3 Research contribution ..... 44
7.4 Product contribution ..... 45
7.5 Limitations and threats to validity ..... 47
7.5.1 Threats to Validity ..... 47
7.5.2 Limitations ..... 47
7.6 Implications ..... 48
8 COMMERCIALISATION AND SUSTAINABILITY ..... 49
8.1 Value of the Component ..... 49
8.2 Target Users and Market Position ..... 50
8.3 Business Model and Feasibility ..... 52
8.4 Sustainability and Maintenance ..... 54
8.5 Contribution to the Sustainable Development Goals ..... 55

9 CONCLUSION AND FUTURE DIRECTIONS ..... 56
9.1 Conclusion ..... 56
9.2 Achievement of objectives ..... 56
9.3 Individual research and product contributions ..... 57
9.4 Future directions ..... 57
REFERENCES ..... 59
GLOSSARY ..... 62
APPENDICES ..... 63
Appendix A Supporting Technical Material ..... 63
A.1 Test catalogue ..... 63
A.2 Retail store use case diagram source, version 1 ..... 64
Appendix B: Disclosure of AI Tool Use ..... 65

# LIST OF FIGURES

**Figure 1.1:** The AutoForge System Diagram ..... 1
**Figure 2.1:** Conceptual framework for the architecture and UI/UX component ..... 11
**Figure 4.1:** The component's place in the complete AutoForge architecture ..... 18
**Figure 4.2:** Architecture plan generation ladder and diagram pipeline ..... 21
**Figure 4.3:** UI/UX Agent pipeline from metadata to approved design ..... 24
**Figure 6.1:** Retail store sequence diagram ..... 36
**Figure 6.2:** Retail store class diagram ..... 37
**Figure 6.3:** Finodil Item Listing use case diagram before and after revision ..... 38
**Figure 6.4:** Retail store page previews: Add Item (left) and List Items (right) ..... 39

# LIST OF TABLES

**Table 2.1:** Design-stage properties of related systems ..... 10
**Table 3.1:** Features whose design artefacts are analyzed ..... 12
**Table 3.2:** Measures used for each research question ..... 14
**Table 3.3:** Configuration parameters of the component ..... 15
**Table 4.1:** Functional requirements of the component ..... 16
**Table 4.2:** Non-functional requirements of the component ..... 17
**Table 4.3:** Main design decisions ..... 18
**Table 4.4:** Sections of the architecture plan ..... 20
**Table 4.5:** Main UML rules checked by the diagram validators ..... 22
**Table 5.1:** Technology stack used by the component ..... 27
**Table 5.2:** Main modules of the component ..... 28
**Table 5.3:** Automated tests for the component ..... 31
**Table 6.1:** Key behaviours confirmed by automated tests ..... 33
**Table 6.2:** Traceability matrix of the retail store plan ..... 34
**Table 6.3:** SRS data requirements and the fields the fallback produced ..... 34
**Table 6.4:** Issues in stored diagrams and whether they were flagged ..... 35
**Table 6.5:** Gaps found in the retail store page design ..... 39
**Table 6.6:** Pipeline with human gates on and off ..... 40
**Table 6.7:** Execution performance of the full pipeline ..... 41
**Table 6.8:** Summary of findings by research question ..... 42
**Table 8.1:** Target users of the component ..... 50
**Table 8.2:** Alternatives for producing a feature design ..... 51
**Table 8.3:** Revenue elements that depend on this component ..... 52
**Table 8.4:** Work needed before a commercial release ..... 54
**Table 9.1:** Achievement of the objectives ..... 56
**Table A.1:** Test files for the component ..... 63
**Table B.1:** Use of generative AI tools ..... 65

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| AC | Acceptance Criterion |
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| CRUD | Create, Read, Update and Delete |
| CSS | Cascading Style Sheets |
| DTO | Data Transfer Object |
| FR | Functional Requirement |
| HTML | HyperText Markup Language |
| JSON | JavaScript Object Notation |
| LLM | Large Language Model |
| NFR | Non-Functional Requirement |
| PNG | Portable Network Graphics |
| QA | Quality Assurance |
| RAG | Retrieval-Augmented Generation |
| REST | Representational State Transfer |
| RQ | Research Question |
| SDG | Sustainable Development Goal |
| SDLC | Software Development Life Cycle |
| SRS | Software Requirements Specification |
| UI | User Interface |
| UML | Unified Modeling Language |

# 1 INTRODUCTION

## 1.1 Overall project context

AutoForge is our team's final-year research project: a multi-agent system that builds features of an e-commerce web application with large language models (LLMs). Instead of turning one prompt straight into code, it works in stages, much as a software team would. A Requirement Agent writes the Software Requirements Specification (SRS), a Domain Agent enriches it with domain knowledge, an Architecture Agent designs the solution, a UI/UX Agent designs the screens, a Coder Agent implements the change, and Security and Quality Assurance (QA) agents check the result. Every agent produces a versioned artefact, and a person can approve it, reject it or ask for changes before the next agent uses it.

LLM can already write a working function or page on request, but a full application depends on a chain of design decisions. Tools that go from prompt to code skip that chain, so when the result is wrong it is hard to tell whether the requirement, the design or the code was at fault. Multi-agent frameworks such as ChatDev [1] and MetaGPT [2] give agents separate roles, but their designs are agreed through agent conversation, not reviewed by a person as separate documents. Figure 1.1 shows the AutoForge pipeline as it appears in the paper.

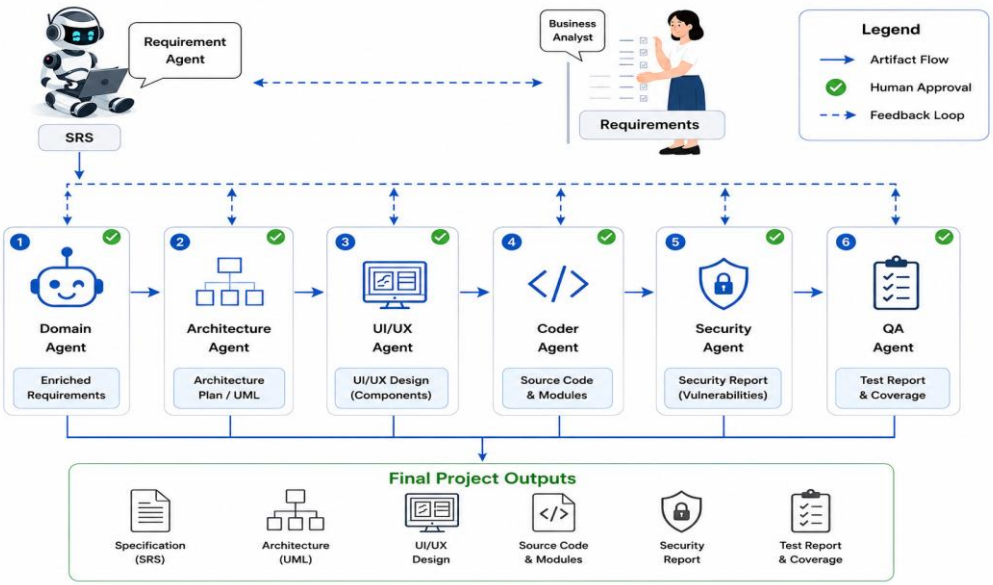



**Figure 1.1:** The AutoForge System Diagram

The project was split into four components, one per team member. My component is the design stage of the pipeline: the Architecture Agent, which produces an architecture plan and three UML diagrams, and the UI/UX Agent, which produces page designs that a reviewer can see and the Coder Agent can follow. The other components cover requirements and orchestration, code generation, and security and quality assurance.

### 1.2 Individual component context

The design stages are where a specification turns into decisions a developer can act on: which endpoints exist, which data entities hold which fields, which classes and modules carry the logic, and which pages the user sees. In AutoForge, the Coder Agent does not invent this structure. It reads the approved architecture plan, including a file-level implementation plan, and the approved page designs, and builds what they describe. Whatever the design stages get wrong, the Coder Agent will faithfully implement.

This gives my component two audiences with different needs. The Coder Agent needs a plan that is complete and machine-readable: every requirement mapped to a design element, every endpoint with a method and a path, and every page with its components and content. The human reviewer needs something they can judge quickly: a readable plan, diagrams in a notation they already know, and screens they can look at rather than imagine. Much of the design work described in this thesis is about meeting both needs from the same artefacts.

Diagrams deserve a special mention. A Unified Modeling Language (UML) diagram is only useful if it follows UML conventions; a use case diagram whose actor is a database, or whose use case is half a sentence, is worse than no diagram, because it looks authoritative. Getting an LLM to produce diagrams that are not just syntactically valid PlantUML but also sensible UML turned out to be one of the hardest parts of the component.

### 1.3 Problem statement

When an LLM produces a design from a specification, three problems appear. First, the design may silently leave out requirements, so a reviewer who does not check every line approves an incomplete plan. Second, generated diagrams are often valid in syntax but wrong in modelling: technical components appear as actors, one action is split into several use cases or several actions are merged into one, and non-functional requirements turn up as messages or classes. Third, generated user interfaces often look finished while showing placeholder content, such as "No data available", or a screen that the model never filled in, so the reviewer cannot judge the real design.

Behind all three is the same issue: a locally hosted model will sometimes return output that is incomplete, malformed or simply wrong, and the stage still has to produce something a person can review. The problem this thesis addresses is therefore: how can the architecture and UI/UX stages of an LLM-based pipeline produce designs that are traceable to the approved specification, follow UML and interface conventions, and can be reviewed and corrected by a person, even when the model's output is unreliable?

### 1.4 Research gap

The systems reviewed in Chapter 2 point to a gap with three parts. MetaGPT [2] produces a design document and diagrams through role agents, but the design is checked by other agents rather than by a deterministic coverage check or a person, and the published work does not report how often the diagrams follow UML conventions. Coding agents such as SWE-agent [3] and OpenHands [4] start from a task that is already specified and designed, so the design step is outside their scope. Traceability research [5], [6] stresses the value of links from requirements into design, yet in the LLM systems we reviewed, links from requirement identifiers to design elements are not reported as a checked property of the output.

Human-in-the-loop work such as HULA [7] shows that engineers value reviewing an agent's plan before code is written. What is missing is a design stage that produces an architecture plan, diagrams and page designs as separate, reviewable artefacts,

checks them against the specification with rules that do not depend on the model, and still gives the reviewer something useful when the model fails.

### 1.5 Aim and objectives

The aim of this work is to design, implement and evaluate the Architecture and UI/UX agents of AutoForge, so that the designs the Coder Agent builds from are traceable, follow modelling conventions, and are approved by a person.

To reach this aim I set four objectives:

1. O1: Build an Architecture Agent that produces a structured architecture plan, including a file-level implementation plan for the Coder Agent, and checks that every requirement in the approved SRS is traced to the plan.
2. O2: Generate use case, sequence and class diagrams through narrow specifications, deterministic modelling and rule-based validation, and render them with PlantUML.
3. O3: Build a UI/UX Agent that plans pages and components, checks them against the SRS, generates each component with realistic content, and assembles previews a person can approve, while keeping a shared design system consistent across features.
4. O4: Evaluate the component through automated tests, the team's case study and the artefacts produced on real features, including where the checks fall short.

### 1.6 Research questions or hypotheses

1. RQ1: Can deterministic coverage validation keep a generated architecture plan traceable to the approved SRS, and what does traceability fail to guarantee?
2. RQ2: Does generating diagrams through narrow specifications, deterministic modelers and UML rule validators produce conventional UML diagrams, and which errors remain?
3. RQ3: Does per-component generation with a content-quality gate and deterministic page assembly produce complete, reviewable page designs?
4. RQ4: Does human review at the architecture and UI/UX stages catch problems before code is written?

### 1.7 Scope limitations and assumptions

The work covers the architecture and UI/UX stages for single features of e-commerce web applications whose target stack is the one the Coder Agent generates: a Next.js App Router front end in TypeScript with MongoDB persistence. The architecture plan is a feature-level plan, not a full system architecture, and the diagrams are limited to use case, sequence and class diagrams. The UI/UX Agent produces static HTML and Tailwind CSS designs with realistic example content; it does not produce working interface code, which is the Coder Agent's job.

The evaluation is limited in scale. The team's case study covers three features, and the detailed artefact analysis in Chapter 6 uses features from two projects. This is enough to show how the mechanisms behave on real input, including their failures, but not to give rates with statistical confidence. Diagram quality was judged by inspection against UML conventions rather than by an independent panel.

I assume that the SRS reaching my agents has been approved by a person, and that a reviewer with some knowledge of software design is available to approve or revise each plan and page design.

### 1.8 Individual contribution

Within the team, my responsibility was the design stage. My contributions are:

- the Architecture Agent's plan structure, prompts and four-rung generation ladder, including the read-only exploration of earlier approved plans and the deterministic fallback;
- the architecture plan validator and the implementation plan that the Coder Agent builds from;
- the use case, sequence and class diagram pipeline: narrow specifications, deterministic modelers, UML rule validators with targeted repair, and PlantUML rendering;
- the UI/UX Agent: page and component metadata, the coverage validator, per-component HTML generation with a content-quality gate, page assembly, preview screenshots and the integration manifest;

- revision through targeted operations for both agents, and the approval cascades and design-system updates that follow a decision.

### 1.9 Thesis structure

Chapter 2 reviews related work and builds a conceptual framework. Chapter 3 describes the research method. Chapter 4 presents the design of the two agents and the decisions behind it, and Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results, including a close look at the artefacts produced for real features, and Chapter 7 discusses what they mean. Chapter 8 considers commercialization and sustainability, and Chapter 9 concludes.

# 2 LITERATURE REVIEW AND RELATED WORK

## 2.1 Review method and scope

I began with two surveys: Hou et al.'s systematic review of LLMs for software engineering [8] and He, Treude and Lo's review of LLM-based multi-agent systems for software engineering [9]. From their references I narrowed the review to the work most relevant to the design stage: multi-agent systems that produce design documents, coding agents that consume a design, human oversight of agent plans, and traceability from requirements into design. Work on testing and security with LLMs [10], [11] was included for one specific lesson, explained in Section 2.3, about verifying model output.

The review shares its sources with the team's research paper. I did not attempt a full systematic review; the aim was to understand what existing systems do at the design stage precisely enough to design my component against the gap.

## 2.2 Domain and theoretical foundations

### 2.2.1 Architecture plans and design views

An architecture plan records the decisions that turn requirements into a buildable structure: the main modules and their responsibilities, the interfaces between them, the data they hold, how they behave in normal and error cases, and how quality requirements such as security and performance are met. A common way to organize such a plan is through views, each answering one kind of question for one kind of reader. I used this idea directly: the AutoForge plan has eight design views, from a context view to a quality-attributes view, so a reviewer can find a particular kind of decision without reading the whole document.

For an agent pipeline, an architecture plan has a second reader, the Coder Agent. A human developer can fill gaps from experience; a coding agent will either guess or stop. So the plan needs a part written for the machine: an ordered list of files to create, the endpoints each implements and the data models they use.

### 2.2.2 UML diagrams

The Unified Modeling Language (UML) provides standard notations for describing software. This component uses three of them. A use case diagram shows who uses the system (actors) and for what goals (use cases), with include and extend relationships for mandatory and optional behaviour. A sequence diagram shows how participants exchange messages over time for one scenario, with fragments for alternatives and options. A class diagram shows the structural elements and their relationships. Each notation has conventions that make it readable: actors are people or external systems, not databases; a use case is a goal, not a step or a sentence fragment; a class is a structural element, not a requirement. These conventions are what my validators check.

### 2.2.3 Traceability into design

Gotel and Finkelstein [5] defined requirements traceability as the ability to follow a requirement backwards to its origin and forwards into design and implementation. Mucha, Kaufmann and Riehle [6] found that most traceability solutions still address one narrow type of link. The link from requirement to design is the first forward link, and in an agent pipeline it can be checked mechanically, because every requirement in the SRS carries a stable identifier. If the plan's traceability matrix does not mention an identifier, the requirement has not been designed for.

### 2.2.4 Human oversight of agent plans

He, Treude and Lo [9] identify human-agent collaboration and the verifiability of agent output as open problems for multi-agent software engineering. Takerngsaksiri et al. [7] report that engineers using HULA, a human-in-the-loop agent framework, valued the chance to review and correct the agent's plan before any code was written. A design stage with an approval gate applies the same idea to architecture and screens. Workflow frameworks such as LangGraph, which Pelluru [12] discusses for production multi-agent systems, make such pauses practical.

### 2.2.5 Design science

My work builds artefacts, two agents, and evaluates them in their intended setting. This places it within the design-science paradigm described by Hevner et al. [13], which asks that the artefact address a real problem, that it be evaluated rigorously,

and that the research add knowledge about the design, including where it does not work.

### 2.3 Related research

ChatDev [1] organises role-playing agents into a "virtual software company" that follows a waterfall-like process. Design happens in conversation between agents playing roles such as chief technology officer and programmer. The design is therefore implicit in the dialogue rather than a document a person can review on its own.

MetaGPT [2] is the closest published system to my component. Its role agents exchange structured documents, and its architect role produces a system design, including data structures and interface definitions and sequence diagrams in a text diagram notation, which the engineer role then implements. MetaGPT shows that structured design documents help coding agents. However, the design is reviewed by other agents, not by a person at a durable checkpoint, and the paper does not report a deterministic check that the design covers every requirement or that its diagrams follow UML conventions.

Many coding agents start from a task that is already specified. SWE-agent [3], OpenHands [4], CodeAct [14], HyperAgent [15] and Agentless [16] work on issues in existing repositories, typically evaluated on SWE-bench [17]. AgentCoder [18] adds test design and execution, and RepoCoder [19] retrieves code from the same repository to guide completion. These systems are strong once the design is settled; they do not produce the design. For AutoForge, they mark the boundary of my component: what I produce is the kind of input they assume.

RepoCoder's idea of using the existing repository as context is relevant in a different way. When a project already has approved features, a new feature's design should be consistent with them. This is the reason the Architecture Agent's first rung reads earlier approved plans, the project manifest and the existing workspace before planning.

Finally, surveys of LLMs in testing [10] and security [11] make a point that applies equally to design: most systems rely on the model's own judgement of its output

rather than on an independent, deterministic check. For my component this suggested a clear division of work. The model proposes; ordinary code decides whether the proposal covers the SRS and follows the notation's rules.

### 2.4 Existing systems products and techniques

Table 2.1 compares the systems most relevant to the design stage on the properties my component is built around. N/R means the property is not reported in the publication.

**Table 2.1:** Design-stage properties of related systems

| System | Design artefact | Coverage check against requirements | UML rule checks | Human gate on design |
|---|---|---|---|---|
| ChatDev [1] | Implicit in agent dialogue | N/R | N/R | N/R |
| MetaGPT [2] | System design document, interfaces, sequence diagrams | N/R | N/R | N/R |
| HULA [7] | Implementation plan | N/R | N/R | Yes (plan review) |
| SWE-agent / OpenHands [3], [4] | None (task given) | N/R | N/R | N/R |
| **AutoForge (our project)** | **Plan with 8 views + implementation plan, 3 UML diagrams, UI page designs** | **Yes, deterministic** | **Yes, deterministic** | **Yes** |

Outside research systems, UI generation tools and design assistants can produce screens from a prompt, and diagram-as-code tools such as PlantUML make diagrams easy to generate and version. My component uses PlantUML for rendering, and Tailwind CSS for page designs, because both are text formats that an LLM can write and that render consistently. The techniques that are less common, and where most of my effort went, are the deterministic steps around the model: a coverage validator, UML modelers and validators, a content-quality check on generated screens, and page assembly in plain code.

## 2.5 Gap synthesis and conceptual framework

The review shows a gap with three parts at the design stage: designs are not checked against every requirement by anything other than another model, generated diagrams are not checked against modelling conventions, and screen designs are not presented as separate artefacts that a person approves before implementation.

Figure 2.1 shows the conceptual framework I used. It connects three inputs (the approved specification, earlier approved designs in the same project, and the shared design system) through three generation mechanisms and three controls to three outcomes. As in the rest of AutoForge, the mechanisms produce content and the controls decide whether it may move forward; the human gate is the final control.

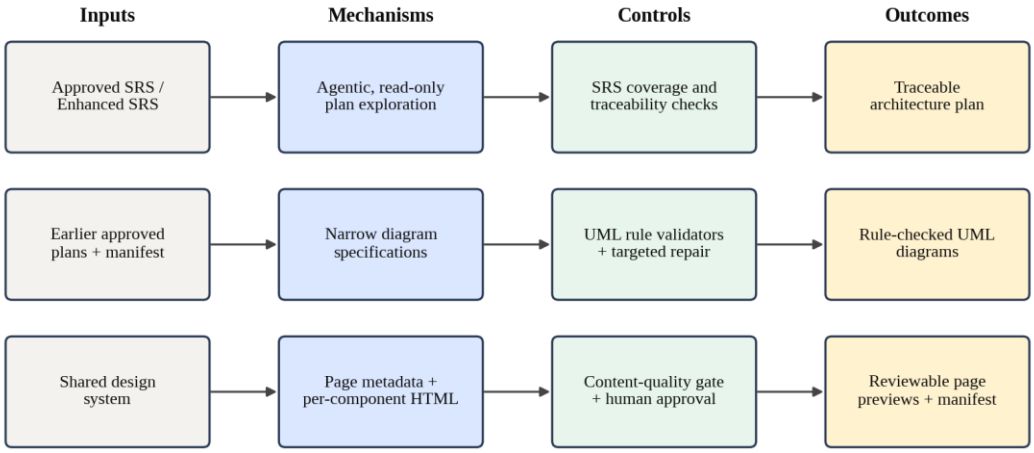



**Figure 2.1:** Conceptual framework for the architecture and UI/UX component

# 3 RESEARCH METHODOLOGY

## 3.1 Research design

This is design-science research in the sense of Hevner et al. [13]. I built two artefacts, the Architecture Agent and the UI/UX Agent, to address the design-stage problem described in Chapter 1 and evaluated them in the pipeline they were built for. No shared benchmark exists for generating a feature-level architecture plan, UML diagrams and page designs from an approved SRS, so the evaluation combines automated tests of the deterministic parts, the team's pipeline case study, and a close reading of the artefacts the agents produced on real features.

Development followed build-and-evaluate cycles between June and August 2026. Each cycle changed one mechanism, tested it against hand-built fixtures, and then ran it on real features. Several design decisions came out of failures found in these runs, for example a use case named "Add List Items" that merged two goals, or a page preview that showed only "No data available". I report these failures as findings, because they explain why the design looks the way it does.

The artefact analysis in Chapter 6 deliberately includes weak outputs. A design stage is judged by what it does when the model's output is poor, not only when it is good, and the stored artefacts show both.

## 3.2 Data participants or study materials

The study materials were the approved specifications given to my agents, the models, and the artefacts the agents produced. Table 3.1 lists the features whose design artefacts are used in this thesis.

**Table 3.1:** Features whose design artefacts are analyzed

| Feature | Project | Source of evidence |
|---|---|---|
| Item Listing (CRUD) | Sample E-commerce | Team case study |
| Authentication and Authorization | Sample E-commerce | Team case study (revision counts, QA, security) |
| Shopping Cart | Sample E-commerce | Team case study (revision counts, QA, security) |

| Feature | Project | Source of evidence |
|---|---|---|
| Item Listing | Finodil (development project) | Stored plan versions 1 and 2 with revision record |
| Add & List Items | Retail Store (development project) | Stored plan, PlantUML sources, page metadata, page previews |

The default model was qwen3-coder served locally through Ollama, the same as for the rest of the pipeline. Some development runs of the UI/UX Agent used llama3, and observations from those runs are reported where they shaped the design. No external participants took part; reviewer decisions were made inside the project team, and no personal data was collected.

To show the diagrams exactly as the agent defined them, the figures in Chapter 6 were rendered from the PlantUML source files the agent saved, using PlantUML with Graphviz, the same renderer the agent uses.

### 3.3 Experimental or study procedure

Four procedures were used.

Component testing. The deterministic parts of both agents, the validators, modelers, patchers, page builder and approval cascades, were tested against hand-built fixtures with no model call. Parts that call a model were tested with stubbed responses, so that the fallback paths could be exercised on purpose, for example a model that never submits a plan or a call that times out.

Pipeline case study. As part of the team evaluation, each case-study feature was carried through all seven stages with human approval. For my stages, the team recorded the plan and diagram versions, the page design versions, the decision at each gate and the reason for each revision.

Artefact analysis. For the two development features, I examined every stored design artefact: the plan JSON, its human approval note, the three PlantUML sources, the page metadata, the integration manifest and the page previews. I compared each against the approved SRS and against UML conventions, and compared the two agents' outputs with each other.

Revision analysis. Where a feature had more than one plan version, I compared the versions and the revision record the agent stores, which lists the reviewer's comment, the changes applied and any requested changes that could not be matched.

### 3.4 Measures and evaluation metrics

**Table 3.2:** Measures used for each research question

| RQ | Measure | How it is obtained |
|---|---|---|
| RQ1 | SRS identifiers traced; design views and sections present; coverage gaps found; quality of the traced content | Validator tests; traceability matrices of stored plans; inspection |
| RQ2 | UML rule violations detected and repaired; violations that remained in stored diagrams | Validator and modeler tests; inspection of PlantUML sources |
| RQ3 | Coverage of actors, UI expectations, stories and visible criteria; placeholder content; preview completeness | Validator and gate tests; stored metadata and previews |
| RQ4 | Revisions requested at the design stages and their causes | Case-study records; stored revision records |

### 3.5 Data analysis

The analysis is descriptive and partly qualitative. The number of features is small, so I report counts directly and do not compute rates or statistical tests. For the artefact analysis, each finding is tied to a specific stored file and, where possible, to the line of code that explains it. This made it possible to separate problems in the model's output from problems in my deterministic code, which matters because only the second kind can be fixed reliably.

### 3.6 Ethics privacy security and responsible AI

The component uses the same locally hosted default model as the rest of AutoForge, so specifications and designs need not leave the machine. The Architecture Agent's exploration tools are read-only: they can list, read and search the project's workspace and earlier plans, but cannot change anything, and they are confined to the project's own workspace. The only tool that changes state is the one that submits the plan, and it only stores the submission for validation.

Page previews are made safe to open: the Tailwind script is embedded in the page rather than loaded from the internet, and every link in a preview is made inert, so a preview cannot navigate away or load outside content. Responsible use also means being honest with the reviewer. When validation finds issues the agent could not fix, they are written into the plan's human approval note in plain English rather than hidden, and the UI/UX Agent has no fallback that would invent a screen design when the model fails.

### 3.7 Reproducibility and research artefacts

All code, prompts, schemas and tests are in the team's shared repository, and every artefact is stored as a numbered version. Table 3.3 lists the settings that affect the results.

**Table 3.3:** Configuration parameters of the component

| Parameter | Value |
|---|---|
| Default LLM | qwen3-coder via Ollama |
| Plan exploration turn limit | 80 steps |
| Diagram exploration turn limit (each) | 20 steps |
| Targeted repairs per diagram type | 2 |
| UI metadata coverage repairs | Up to 4 |
| Component content-quality repairs | Up to 2 |
| Diagram renderer | PlantUML (Java) with Graphviz |
| Preview renderer | Playwright headless Chromium |
| Generated stack assumed | Next.js App Router, TypeScript, MongoDB |

Model output is not deterministic, so a repeated run will not produce identical plans or pages. The validators, modelers, page assembly and approval behaviour are deterministic by design and give the same result for the same input.

# 4 PRODUCT AND SOLUTION DESIGN

## 4.1 Requirements

The requirements for my component came from the team's pipeline objectives, from what the Coder Agent needs in order to build a feature, and from problems found while testing on real features. Table 4.1 lists the functional requirements and Table 4.2 the non-functional ones.

**Table 4.1:** Functional requirements of the component

| ID | Requirement |
|---|---|
| C2-FR-01 | Run only when an approved SRS exists, and use the approved Enhanced SRS instead when there is one |
| C2-FR-02 | Produce an architecture plan with fixed sections, eight design views and a traceability matrix |
| C2-FR-03 | Include a file-level implementation plan and implementation tasks for the Coder Agent |
| C2-FR-04 | Check that every FR, AC, VR and NFR identifier of the SRS appears in the plan's traceability matrix |
| C2-FR-05 | Take earlier approved plans of the same project into account when planning a new feature |
| C2-FR-06 | Generate use case, sequence and class diagrams, validate them against UML rules and render them as PNG |
| C2-FR-07 | Revise a plan by applying only the reviewer's requested changes, and report anything not matched |
| C2-FR-08 | Run the UI/UX stage only when the architecture plan is approved |
| C2-FR-09 | Plan pages and components that cover every actor, UI expectation, user story and screen-related acceptance criterion |
| C2-FR-10 | Generate each component with realistic content and assemble pages that open without a network connection |
| C2-FR-11 | Reuse approved components from earlier features instead of regenerating them |
| C2-FR-12 | Produce an integration manifest describing routes, navigation and components for the Coder Agent |
| C2-FR-13 | Update the project's shared design system only when a page design is approved |
| C2-FR-14 | Revise page designs, optionally limited to the pages the reviewer selects |

**Table 4.2:** Non-functional requirements of the component

| ID | Quality | Requirement |
|---|---|---|
| C2-NFR-01 | Reliability | A failed or malformed model response must never stop the architecture stage from producing a reviewable plan |
| C2-NFR-02 | Honesty | Issues the agent could not fix must be shown to the reviewer, not hidden |
| C2-NFR-03 | Generality | Validators and modelers must contain no feature-specific rules (no login, cart or payment cases) |
| C2-NFR-04 | Safety | Planning tools must be read-only; previews must not load or navigate to outside content |
| C2-NFR-05 | Consistency | Diagram names must agree with each other and features must share one design system |
| C2-NFR-06 | Testability | Deterministic logic must be testable without a model |
| C2-NFR-07 | Responsiveness | Both agents must stream progress to the interface during long runs |

### 4.2 Component architecture within the complete solution

Figure 4.1 places my component in the complete system. The orchestrator runs the Architecture Agent once the SRS, and the Enhanced SRS if there is one, have been approved, and runs the UI/UX Agent once the architecture plan has been approved. Both are hard gates: the pipeline stops after each until a person decides. The Architecture Agent uses PlantUML to render diagrams, and the UI/UX Agent uses Playwright to take preview screenshots. Both write versioned artefacts and read the project's cross-feature memory: the project manifest, which records the routes, endpoints and models built so far, and the design system, which records shared components and tokens.

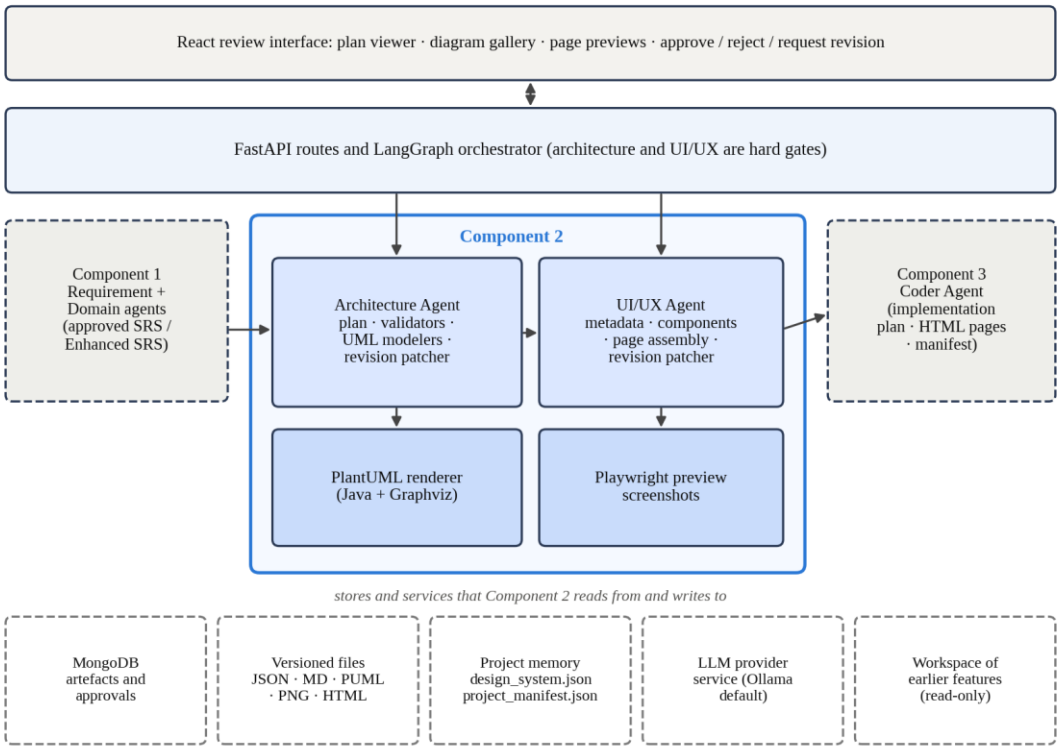



**Figure 4.1:** The component's place in the complete AutoForge architecture

The Coder Agent is the main consumer of my outputs. It reads the approved plan, in particular its implementation plan, and uses the approved page designs and the integration manifest as a visual and structural reference. The boundary is therefore the approved artefact, as elsewhere in AutoForge: neither of my agents calls the Coder Agent, and the Coder Agent never calls them.

### 4.3 Design alternatives and decisions

**Table 4.3:** Main design decisions

| Decision | Alternative | Reason for the choice |
|---|---|---|
| Agentic, read-only exploration as the first rung | Single prompt with everything pasted in | Lets the model look up earlier plans and code only when it needs them; any failure falls back to the proven ladder |
| Deterministic SRS-derived fallback plan | Fail the stage when the model fails | The UI/UX and Coder stages are blocked until a plan exists; a reviewer can judge and revise a weak plan |
| Diagrams from narrow specifications + modelers | Model writes PlantUML directly | Valid PlantUML is not the same as sensible UML; modelers apply the rules every time |

| Decision | Alternative | Reason for the choice |
|---|---|---|
| Sequence before class diagram | Generate both independently | The class step sees the finalized sequence names, so the two diagrams use the same names |
| Issues reported in the approval note | Block the run on any validator failure | Heuristic validators can be wrong; a person can judge a borderline case in seconds |
| No fallback for UI/UX | Template screens when the model fails | A guessed screen design misleads the reviewer more than an honest failure |
| Static HTML + Tailwind components | React components rendered in the preview | Static markup has no state branches that can fall into an empty or unknown state |
| Page assembly in plain code | Model writes the full page | Assembly cannot fail or invent content; one broken component can be regenerated alone |
| Design system updated on approval | Update after every run | A rejected design cannot change the look of later features |

Two of these decisions were reversals. The first version of the UI/UX Agent generated React components and rendered them live in the preview. In a real run on the Item Listing feature, a page showed only "Unknown state." because the generated component handled three of its four declared states and the preview picked the fourth. Switching to static HTML with realistic content baked into the markup removed the whole class of problem, because there are no states left to pick from. The second reversal concerned revision. The Architecture Agent originally asked the model to retype the entire plan with the requested change; a stored plan's revision record showed that a parse failure had made the fallback copy the old plan unchanged, silently dropping the reviewer's request. Revision now uses targeted operations, as described in Section 4.4.4.

## 4.4 Detailed design

### 4.4.1 Architecture plan structure

The architecture plan is a JSON document with fifteen required sections, listed in Table 4.4, plus an implementation plan. The eight design views each answer one kind of question, which keeps a long plan navigable for a reviewer. The plan is also converted to Markdown and to a PDF for reading.

**Table 4.4:** Sections of the architecture plan

| Group | Sections |
|---|---|
| Context | document_control, feature_overview, requirement_interpretation, architecture_approach |
| Design views | context, logical, interface, data, behaviour, error handling, security and authorisation, quality attributes |
| Delivery | frontend_architecture_plan, backend_architecture_plan, validation_plan, coder_implementation_tasks, implementation_plan |
| Links | traceability_matrix (one row per SRS identifier and other traced item) |
| Risk and review | assumptions, constraints, risks, dependencies, human_approval_note |

The implementation plan is written for the Coder Agent. It lists the back-end files with the endpoints each implements, the data models with their fields, the front-end pages and the services they call, and an ordered list of steps. File paths follow the conventions the Coder Agent's own planner uses for a Next.js App Router project, such as app/api/items/route.ts for a Route Handler, and a collection endpoint and an item endpoint always map to separate files because Next.js routes by folder. When the model leaves the implementation plan out, the agent derives one mechanically from the plan's own interface and data views, so every plan the Coder Agent receives has one.

### 4.4.2 Generation ladder and coverage validation

Figure 4.2 shows how a plan is produced. The first rung is an agentic loop in which the model can use six tools: list, read and search files in the project's workspace, read the project manifest, read an earlier feature's approved plan by name, and submit its plan. All but the last are read-only. A project's first feature has no workspace yet, so the workspace tools return a plain message saying so instead of failing. The loop is limited to 80 steps; if it ends without a valid submission, the agent moves to the second rung, a single schema-constrained call with earlier plans summarised in the prompt. If that output fails, a third rung asks the model to repair the JSON, and if that also fails, the fourth rung builds the plan deterministically from the SRS.

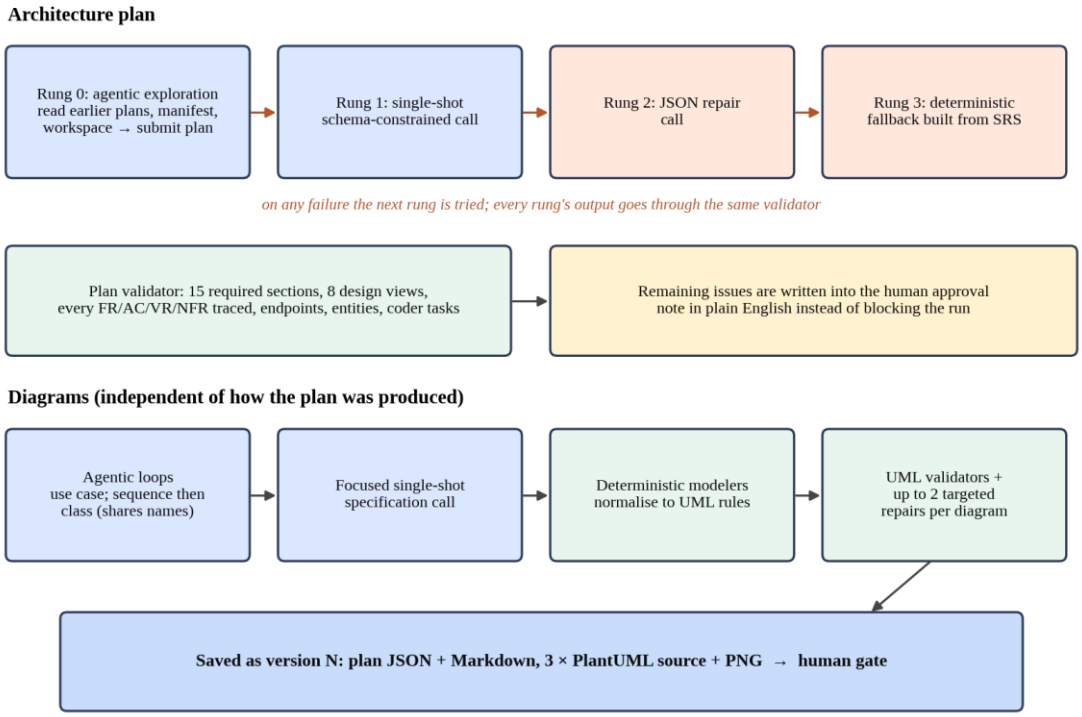



**Figure 4.2:** Architecture plan generation ladder and diagram pipeline

Every rung's output passes through the same validator. It checks that all fifteen sections and all eight design views are present; that every functional requirement, acceptance criterion, validation rule and non-functional requirement identifier in the SRS appears in the traceability matrix; that every API expectation in the SRS has an endpoint in the interface view; that data requirements have data entities; that non-functional requirements are addressed in the quality view; and that every functional requirement is covered by at least one Coder Agent task. A transport failure, such as a timeout with an empty error message, is treated like an unusable response, so it moves the run to the next rung instead of crashing it.

If even the fallback plan fails a heuristic check, the run is not stopped. The plan has to exist before the UI/UX and Coder stages can run, so the agent writes the remaining issues into the plan's human approval note as short plain-English lines and lets a person decide. The same happens for diagram issues that survive repair.

### 4.4.3 UML diagram generation

Diagrams are generated separately from the plan, so their quality does not depend on which rung produced it. The model is never asked to write PlantUML. Instead it

writes a small JSON specification: actors, use cases and relationships for the use case diagram; participants and ordered interactions for the sequence diagram; classes, attributes, operations and relationships for the class diagram. Each specification has three ways to be produced: an agentic loop of up to 20 steps with read and validate tools, a focused single-shot call, or, if both give nothing, the modeler's own deterministic default. The sequence specification is produced first, and the class step is given the finalised sequence names, which is what keeps the names in the two diagrams consistent.

Deterministic modelers then turn each specification into the final diagram model, applying feature-independent UML rules: actors are external roles or systems; the main use case represents the feature's goal; mandatory supporting behaviour becomes an include relationship and optional or error behaviour an extend relationship; sequence diagrams show the main success flow first and alternatives in alt fragments; and constraints, risks and non-functional requirements never become use cases, messages or classes.

Three validators check the result. Table 4.5 lists the main rules. When a diagram fails, the agent makes a small, targeted repair call that passes the validator's message back to the model and asks it to fix only the flagged issues, at most twice per diagram, rather than regenerating the whole plan. The final models are converted to PlantUML and rendered to PNG.

**Table 4.5:** Main UML rules checked by the diagram validators

| Diagram | Rules |
|---|---|
| Use case | System boundary, at least one actor, use case and relationship; no technical components as actors; no generic, over-long or sentence-fragment names; no one action split into parallel steps; no name merging two distinct capabilities; correct include and extend direction; traceability to SRS identifiers; nothing from the SRS's out-of-scope list |
| Sequence | Participants and at least three messages; an actor and a boundary or control lifeline; messages only between existing participants; balanced alt and else fragments; no non-functional or architecture terms as messages; no out-of-scope behaviour |
| Class | At least one class; no duplicate classes; no requirements, risks or use case phrases as classes; attributes and operations as lists; relationships only between existing classes with valid types |

### 4.4.4 Revision

A reviewer's revision request is turned into a short list of operations (add, remove, modify or set) aimed at a dotted path in the plan, for example design_views.context_view.external_systems. Python applies them. Because the plan's lists are keyed differently, a file by its path, an endpoint by its method and path, a task by its task identifier, the patcher looks at the shape of the list it finds at the path and matches items by identifier-like key first, then by exact text, then by a distinctive substring. Operations that cannot be matched are reported in the plan's revision record rather than dropped. After a revision the diagrams are regenerated through the focused single-shot route, since a person is waiting and the longer agentic loop would be a poor trade.

### 4.4.5 UI/UX page and component design

Figure 4.3 shows the UI/UX pipeline. The agent first loads the approved SRS (or Enhanced SRS), the approved architecture plan and the project's design system, and makes one model call to produce ui_metadata_json. This lists the pages, each with its route, actors, the requirement identifiers it covers, its interaction states and its components. Each component lists the specific content it displays, grounded in the data entity's real fields, and says whether it reuses a component from the design system or why a new one is needed. The model also chooses one colour family for the whole feature, so every component uses the same accent colour.

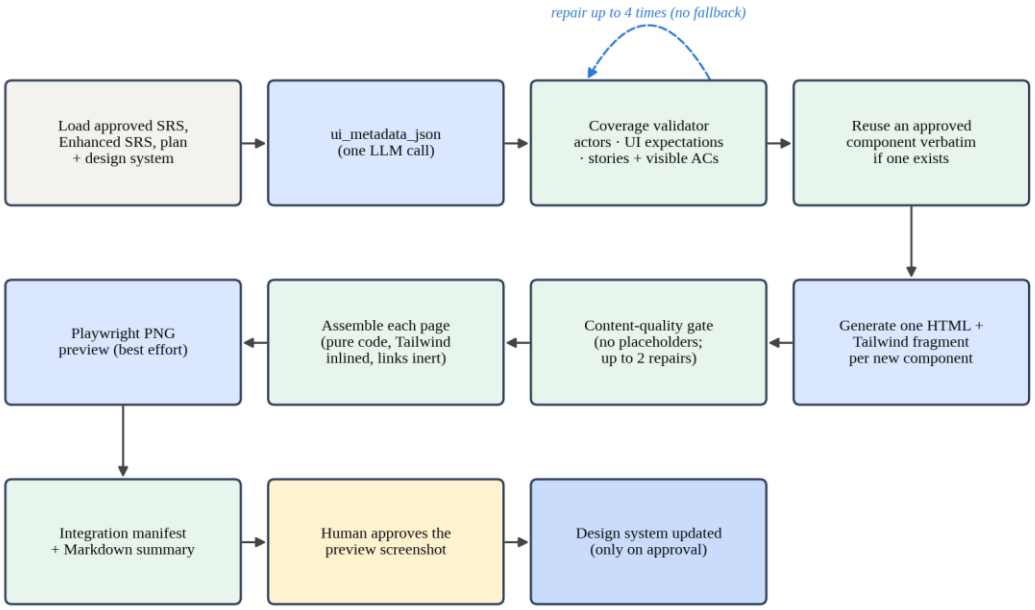



**Figure 4.3:** UI/UX Agent pipeline from metadata to approved design

A coverage validator then checks that every user role appears on at least one page, every UI expectation is covered by a component, and every user story and every acceptance criterion whose text implies a screen (words such as display, message, redirect, form or button) is listed by a page. It also rejects content lists that simply echo the prompt's own example text, a failure seen in a real run. If coverage fails, the validator's message is sent back for a targeted repair, up to four times. There is deliberately no deterministic fallback: a guessed screen design would mislead the reviewer more than an honest failure.

Components are then generated one at a time, each as a fragment of static HTML with Tailwind CSS classes and realistic example content. If a component of the same name has already been approved in another feature of the project, its approved markup is reused word for word instead of generating a new copy. The model returns the HTML after a plain text marker rather than inside JSON, because multi-line HTML inside a JSON string is a common cause of escaping errors. Each fragment passes a content-quality gate that rejects placeholder text such as "No data available", "Unknown state", "Coming soon", "lorem ipsum" or "TODO", with up to two targeted repairs. Because components are generated separately, one broken component can be repaired without touching the others.

Pages are assembled in plain code. Each page joins its component fragments into a complete HTML document with the Tailwind script embedded, so the file opens without a network connection, and with a small script that makes every link inert. That script fixes a real bug: a link with href="#" inside the preview frame used to load the host application inside its own preview. Playwright then takes a PNG screenshot of each page for the gallery, on a best-effort basis; the primary preview is the HTML itself. Finally, the agent builds an integration manifest for the Coder Agent, listing each page's route, navigation entry, components and content, and a Markdown summary for the reviewer.

### 4.4.6 Approval, cascades and the design system

Each stage produces several artefacts that belong together. For the architecture stage these are the plan JSON, its Markdown version and the six diagram files; for the UI/UX stage they are the metadata, manifest, Markdown, component and page files and the screenshots. A reviewer approves one artefact, the plan or a preview screenshot, and the approval service applies the same decision to the rest of that version. Approving a newer version returns the previously approved one, and its siblings, to pending. The cascade never touches other versions or unrelated artefact types, and each cascaded artefact records that its approval came from the cascade.

Approving a UI/UX design is also the only thing that updates the project's design system. New components and tokens from the approved version are merged into design_system.json, so a later feature can reuse them. A rejected or revised design leaves the design system untouched. UI/UX revision uses targeted operations on components, matched by page and component name, and when the reviewer selects specific pages, any operation that would touch another page is rejected and reported.

## 4.5 Standards and professional practice

The diagrams follow UML notation for use case, sequence and class diagrams, and the validators encode its common conventions. The plan's design views follow the long-standing practice of describing an architecture through several views for different concerns, and the traceability matrix links each design element to stable requirement identifiers, which is the forward link that traceability research [5], [6]

treats as essential. Endpoints follow REST conventions, and file paths follow the conventions of the Next.js App Router.

On the engineering side, all inputs and outputs are typed schemas, validators and modelers contain no feature-specific rules, agentic tools follow least privilege by being read-only, and every artefact is versioned. Model output is treated as untrusted input that is parsed, validated and, where necessary, rejected or flagged before anything downstream relies on it.

# 5 IMPLEMENTATION INTEGRATION AND TESTING

## 5.1 Development environment and technology stack

Both agents are Python packages inside the team's FastAPI backend. Table 5.1 lists the main technologies and the reason for each.

**Table 5.1:** Technology stack used by the component

| Concern | Technology | Why it was used |
|---|---|---|
| Web API and schemas | FastAPI, Pydantic | Typed requests and responses; streaming progress to the interface |
| Agentic loops | LangChain create_agent with LangGraph | Tool-using loops with a step limit for plan and diagram exploration |
| Default LLM | qwen3-coder via Ollama | Local model that followed the JSON schemas reliably |
| Diagram rendering | PlantUML (Java) with Graphviz | Text-based UML that renders consistently and can be versioned |
| Page styling | Tailwind CSS (embedded script) | Utility classes an LLM writes well; no build step for previews |
| Preview screenshots | Playwright, headless Chromium | Reliable screenshots of already-painted HTML |
| Artefacts | MongoDB records + versioned files | Shared artefact store and approval records of the team |
| Frontend | React 19, Vite, Tailwind CSS 4, TanStack Query | The team's shared interface stack |
| Testing | pytest | Fixture-based tests of deterministic logic and stubbed model calls |

## 5.2 Component implementation

Each agent follows the team's package layout: an entry point, a prompt module, a schema module and stage-specific modules. Table 5.2 lists the main modules and their size. The Architecture Agent package has about 13,000 lines and the UI/UX Agent package about 3,300, which reflects where the difficulty lay: getting reliable diagrams took far more code than getting reliable pages.

**Table 5.2:** Main modules of the component

| Module | Lines | Responsibility |
|---|---|---|
| architecture_agent/agent.py | 4,280 | Run and revise flows, ladder, diagram tiers, fallback plan, implementation plan |
| architecture_agent/prompt.py | 1,642 | Plan, diagram, repair and revision prompts |
| architecture_agent/usecase_modeler.py + validator | 1,889 | Use case normalisation and UML rule checks |
| architecture_agent/sequence_modeler.py + validator | 1,114 | Sequence model and fragment checks |
| architecture_agent/class_modeler.py + validator | 1,089 | Class model and structural checks |
| architecture_agent/sds_validator.py | 335 | Plan structure and SRS coverage |
| architecture_agent/revision_patcher.py | 316 | Targeted plan operations |
| architecture_agent/tools.py, diagram_tools.py | 525 | Read-only planning tools and diagram tools |
| uiux_agent/agent.py | 1,458 | Run and revise flows, component reuse, artefacts, design system patch |
| uiux_agent/prompt.py | 439 | Metadata, component, repair and revision prompts |
| uiux_agent/metadata_validator.py | 222 | Coverage and placeholder checks |
| uiux_agent/component_generator.py | 203 | Per-component HTML generation and content gate |
| uiux_agent/revision_patcher.py | 292 | Targeted component operations with page scoping |
| uiux_agent/page_html_builder.py, preview_renderer.py | 213 | Page assembly and screenshots |

### 5.2.1 Architecture Agent

The agent's run function checks that an approved SRS exists, loads the approved Enhanced SRS if requested and available, loads every other feature's latest approved plan and the project manifest, and then calls the generation ladder. The ladder shares one small state dictionary across its rungs, so an expensive agentic diagram attempt is made at most once per run and, if it succeeded on one rung, is reused on the next instead of being thrown away.

The planning tools for the first rung are built separately from the Coder Agent's tools on purpose. The Coder Agent's tool builder creates the project's workspace as a side effect, which would be wrong for a stage that runs before any code exists. Workspace paths are resolved inside the project's workspace, and a path that tries to escape it is refused. The submit tool does not return the plan as text; it writes the JSON argument into a captured dictionary, which the agent reads after the loop, so there is no free-text chat output to parse.

The fallback plan is built from the SRS section by section: endpoints from API expectations, data entities from data requirements, behaviour from functional requirements and acceptance criteria, error handling from validation rules, the quality view from non-functional requirements, and a traceability row for every identifier. Two parsers turn SRS text into structure. One reads an API expectation such as "POST /api/items" into a method and a path; before it existed, every endpoint defaulted to GET and four CRUD endpoints collapsed into four identical GETs. The other reads a data requirement written as "price (number, required)" into a field name, type and required flag, and consolidates all data requirements into one entity instead of one entity per field. When a data requirement does not match that shape, the fallback infers field names from its words. Chapter 6 shows what this does to a requirement written in a different format.

### 5.2.2 Diagram pipeline

The three modelers and three validators are independent classes, each with no knowledge of any particular feature. The use case validator is the largest, because use case naming produced the most varied failures. Two of its checks pull in opposite directions. One catches a single action split into several parallel steps under the same use case, for example "Validate Email", "Validate Password" and "Validate Credentials" for one login. The other catches a single name that merges two goals, such as "Add List Items" for a feature that both adds and lists items. A third check catches names cut from the middle of a sentence, such as "A Task The Can", by looking for names made mostly of articles and pronouns.

The PlantUML builders emit plain source files, and a small renderer runs PlantUML, first as an installed command and otherwise from the jar file named in the configuration, to produce each PNG.

### 5.2.3 UI/UX Agent

The UI/UX Agent's run function checks for an approved SRS and an approved architecture plan, loads the design system and calls, in order, metadata generation with coverage repair, component generation, page assembly and rendering, the integration manifest builder and the Markdown builder. Preview rendering uses Playwright's synchronous API, which cannot run inside the server's event loop, so it is called in a worker thread. A failed screenshot is logged and skipped rather than failing the run, since the HTML page is the primary preview.

Component reuse looks up an approved fragment of the same name in any feature of the same project and returns the latest approved version unchanged. This was added after a second-feature test in which the Signup feature correctly marked LoginForm as reused, but the agent generated a fresh LoginForm anyway, so the flag was only a label.

### 5.2.4 Interface support

On the frontend, the component has chat panels for both agents, a viewer for the architecture plan, a gallery of the rendered diagrams, a page-preview panel that shows each assembled page live in a sandboxed frame, and a list that groups UI/UX artefacts by version. Reviewers can also download all preview images of one version as a single zip file.

## 5.3 Integration with the other components

Integration with the requirement stage is through the approved SRS and Enhanced SRS. A helper prefers the Enhanced SRS whenever one has been approved, both in the single-shot prompt and in the agentic prompt, and tests check that the two prompts make the same choice.

Integration with the Coder Agent happens through three artefacts. The implementation plan gives the files, endpoints and models to build and the order to build them in; tests confirm that the Coder Agent's planner prompt includes it and

leaves it out for old plans that do not have one. The page designs and the integration manifest give the routes, navigation entries and the content each page should show. The project manifest, which the Coder Agent updates after each merged feature, flows back into the next feature's architecture plan.

Integration also exposed mismatches between conventions. The Requirement Agent's own prompt shows data requirements in the form "name (string, required)", and the fallback plan's parser was written for that form. In practice, analysts also answered in the form "name: string", and from 20 August the requirement quality gate accepted that form as a structured field, so such entries reached the SRS unchanged. The Architecture Agent's parser did not recognise them. Section 6.2.1 shows the consequence. Mismatches like this are easy to miss, because each agent's own tests pass.

### 5.4 Verification and software testing

Table 5.3 summarises the automated tests for the component. Appendix A lists every test file.

**Table 5.3:** Automated tests for the component

| Area | Test files | Test functions |
|---|---|---|
| Architecture plan, ladder, tools and revision | 11 | 91 |
| UML modelers, validators, builders, tools and repair | 18 | 165 |
| Architecture approval cascade | 1 | 6 |
| UI/UX metadata, components, pages, reuse and revision | 10 | 92 |
| UI/UX approval cascade | 1 | 7 |
| **Total** | **41** | **361** |

Several tests target the fallback paths directly, because those paths only run when something has gone wrong and are otherwise rarely exercised. They check that a plan submitted through the exploration tools is used without any further model call; that a loop which never submits, or hits its step limit, falls back cleanly; that a timed-out call moves to the repair rung and then to the fallback; and that the streaming version

of the run ends with a final event rather than an error. On the UI/UX side, tests check that a component failing the content gate once is repaired, that one failing every attempt raises a clear error, and that approving a design updates the design system while rejecting it does not.

The collected 382 test cases whose names refer to the architecture or UI/UX stages, counting parameterized cases. The failures recorded for this component in that run's cache all belong to tests that have since been renamed or removed, so no current test of the component is recorded as failing. Like the rest of the backend suite, some tests need a running MongoDB database through the shared test setup.

Tests cannot show whether a generated design is good, only whether the rules hold. The quality of the designs was checked by reviewing the stored artefacts at each milestone, which is how the problems reported in Chapter 6 were found.

### 5.5 Deployment and operation

The component is deployed with the AutoForge backend. In addition to the team's requirements, it needs Java with the PlantUML jar and Graphviz for diagram rendering, and Playwright with Chromium for screenshots. The Tailwind script is stored inside the repository and embedded into each page, so previews need no internet access. Each project's design system and manifest are stored as JSON files beside its artefacts, and both persist across restarts.

# 6 RESULTS AND VALIDATION

This chapter reports the results in the order of the research questions, after summarising what the automated tests established. Pipeline level results come from the team's evaluation in our research paper. The rest come from the artefacts my agents stored for two features developed during the project: "Add & List Items" in a Retail Store project and "Item Listing" in a project called Finodil. These artefacts include weak outputs as well as good ones, and I report both.

## 6.1 Research results

Table 6.1 lists the behaviours that the rest of the pipeline relies on and the tests that check them. All were passing in the last recorded run. They show that the deterministic rules hold; they do not show that the designs are good, which is the subject of Section 6.2.

**Table 6.1:** Key behaviours confirmed by automated tests

| Behaviour | Test file | Result |
|---|---|---|
| A plan submitted through the exploration tools is used with no further model call | architecture_agent_exploration | Pass |
| A loop that never submits, or hits its step limit, falls back cleanly | architecture_agent_exploration | Pass |
| A timed-out call moves to the repair rung and then to the fallback | architecture_agent_transport_errors | Pass |
| Several data requirements become one entity; CRUD endpoints keep their real methods | architecture_data_requirements_consolidation | Pass |
| Revision operations change only their target; unmatched ones are reported | architecture_revision_patcher | Pass |
| Fragmented, conjoined and sentence-fragment use case names are rejected | architecture_usecase_validator_quality | Pass |
| Approving a plan approves its Markdown and six diagram files, and nothing else | approval_architecture_cascade | Pass |
| An approved component is reused word for word, from the same project only | uiux_component_reuse | Pass |
| Page-scoped revisions cannot touch other pages | uiux_revision_patcher | Pass |
| The design system changes on approval only | approval_uiux_cascade | Pass |

## 6.2 Product and technical results

### 6.2.1 Traceability of architecture plans (RQ1)

The retail store plan, version 1, is a useful test of RQ1 because it was produced by the deterministic fallback rung. Its endpoint purposes all read "Support the Add & List Items feature." and its entity purpose reads "Core data entity for the Add & List Items feature.", which is the fallback's own template text. In other words, the model had failed on every earlier rung, and this is what the safety net produced. Table 6.2 shows its traceability matrix.

**Table 6.2:** Traceability matrix of the retail store plan

| Traced item type | Rows | Required by the validator |
|---|---|---|
| Functional requirements (FR-001, FR-002, FR-DOM-001) | 3 | Yes |
| Acceptance criteria | 5 | Yes |
| Validation rules | 4 | Yes |
| Non-functional requirements | 3 | Yes |
| Data requirements, constraints, UI and API items, risks, dependencies | 21 | No (traced for completeness) |
| **Total** | **36** | |

Every identifier the validator requires is traced, including FR-DOM-001, the detail-view requirement that the Domain Agent added, which shows that the Enhanced SRS was used. All six Coder Agent tasks name their related requirements, and the implementation plan lists a Route Handler at app/api/items/route.ts, a model file, a page and an ordered list of five build steps. By the validator's measure, the plan is complete.The data entity tells a different story. The SRS wrote its data requirements in the form "name: type", which the fallback's field parser does not recognise, so it fell back to inferring field names from the words of each requirement. Table 6.3 shows the result.

**Table 6.3:** SRS data requirements and the fields the fallback produced

| SRS data requirement | Fields produced (all String, none required) |
|---|---|
| image: file upload (stored and served back as an image URL) | image, fileUpload, storedAndServedBackAsAnIm, ageURL |
| name: string | name, string |

| SRS data requirement | Fields produced (all String, none required) |
|---|---|
| description: text (optional) | description, text, optional |
| price: number | price, number |
| quantity: number | quantity, number |
| createdAt: date (set automatically when the item is created) | createdAt, date, setAutomaticallyWhenTheItem, isCreated |

Six real fields became seventeen, with fields called string, number and optional, a field name cut in half (storedAndServedBackAsAnIm and ageURL), price and quantity typed as text, and no field marked as required. The same entity appears in the implementation plan's model and in the class diagram (Figure 6.2). Nothing in the validator objected, because it checks that a data entity exists, not what is in it. The Finodil Item Listing plan shows the same pattern on a smaller scale, with extra fields named itemsTable and string.

The answer to RQ1 is therefore split. Deterministic coverage validation did keep the plan traceable: no SRS identifier went missing, even on the fallback path. But traceability shows that each requirement was planned for, not that it was planned well. A reviewer approving this plan on the strength of its traceability matrix would pass a broken data model to the Coder Agent.

### 6.2.2 Quality of the generated UML diagrams (RQ2)

The tests show that the validators catch each rule violation they were written for. The stored diagrams show which errors reach the reviewer anyway. Table 6.4 summarises what I found in the PlantUML sources of the two development features, and whether a validator flagged it.

**Table 6.4:** Issues in stored diagrams and whether they were flagged

| Diagram | Issue | Flagged |
|---|---|---|
| Retail use case v1 | One use case, "Add List Items", merging two goals | Yes, in the approval note |
| Retail use case v1 | External actor "Image Hosting Solution To", cut from the dependency sentence | No |
| Retail sequence v1 | Unrelated SecurityTokenProvider asked to "Generate required output credential/token" | No |

| Diagram | Issue | Flagged |
|---|---|---|
| Retail sequence v1 | Listing items placed in an "Optional or alternative user flow" fragment | No |
| Retail class v1 | Request and response DTOs with a single attribute "field: String"; seventeen-field entity | No |
| Finodil use case v1 | External actor "No External Dependencies Were", cut from a plan note | Revised by reviewer |
| Finodil use case v2 | Duplicate actors "Logged In User" and "Logged In Users", one unconnected | No |
| Finodil use case v2 | Interface states and domain notes as use cases ("Show Loading Indicator", "Use Stock Movement Logs") | No |
| Finodil use case v2 | Use case diagram missing traceability for AC-003 | Yes, in the approval note |

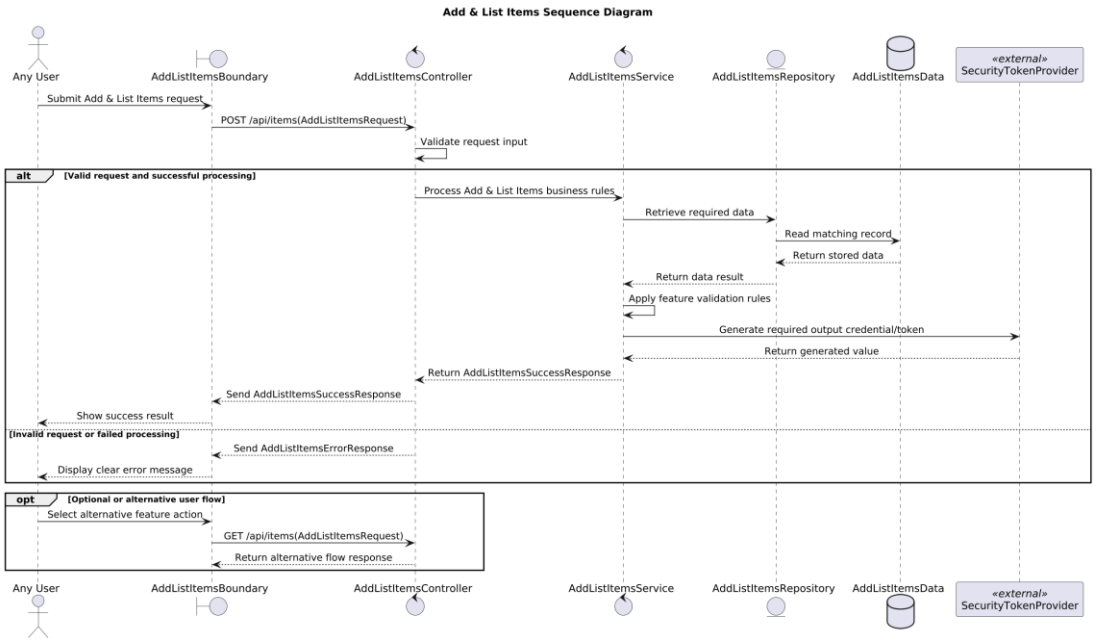



**Figure 6.1:** Retail store sequence diagram

Figure 6.1 shows the retail store sequence diagram. Structurally it follows the rules: an actor, a boundary, controllers, a repository and a data store, ordered messages, and balanced alt, else and opt fragments. In content it is generic. Its messages could belong to any feature, it reads a "matching record" while adding an item, and it calls a security token provider that has nothing to do with this feature. The class diagram in Figure 6.2 has a sensible layered shape, with a controller, a service, a repository,

an entity and data transfer objects, but the objects themselves carry placeholder attributes and the entity carries the fields from Table 6.3.

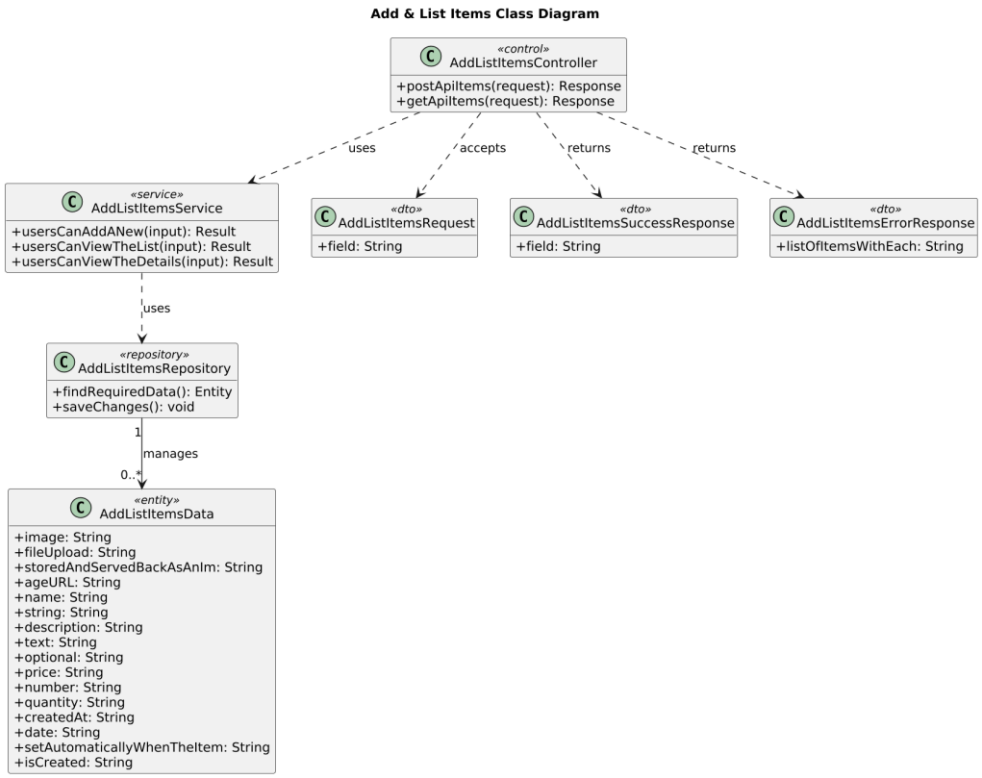



**Figure 6.2:** Retail store class diagram

The Finodil use case diagram shows what a revision can and cannot fix (Figure 6.3). The reviewer asked to "enhance the use case diagram more and remove the <<external system>> actor". The revision record shows that the agent removed the note from which the false actor had been made, and version 2 no longer has it. But the regenerated diagram brought new problems: a second, unconnected actor with a plural name, interface states such as "Display Empty State" as use cases, and four included use cases that are implementation or domain notes rather than user goals, such as "Use Stock Movement Logs". The only issue the validators reported for version 2 was a missing trace to AC-003.

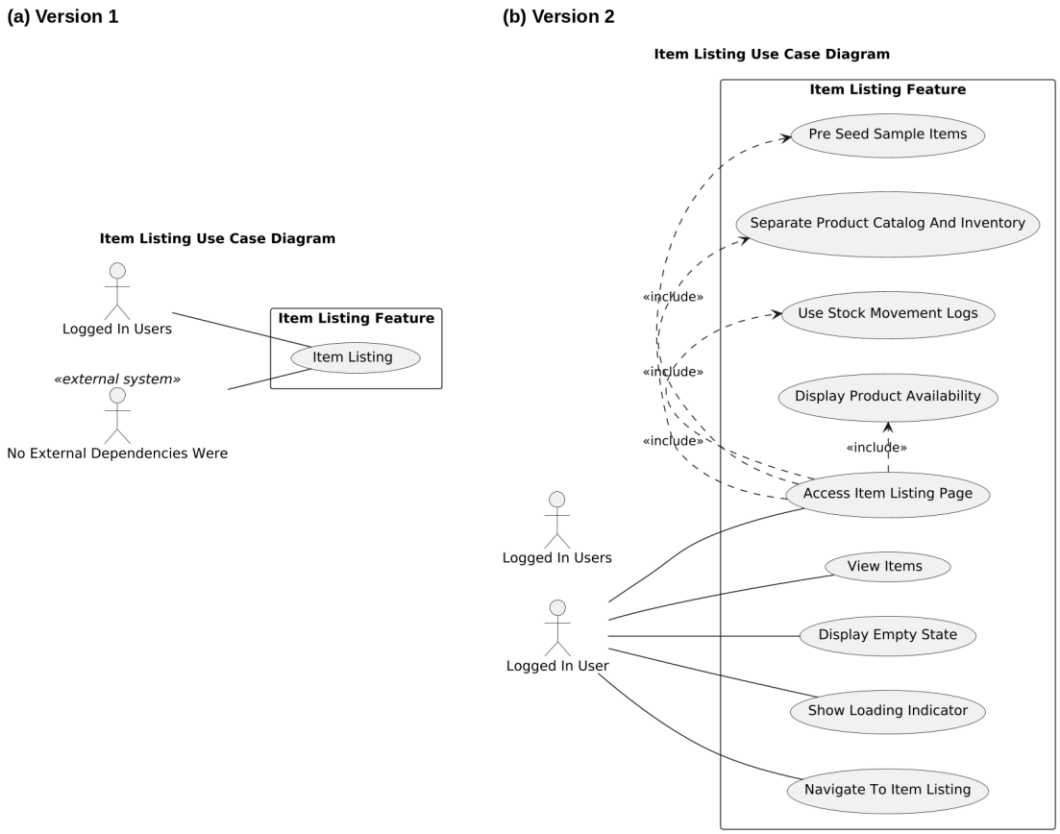



**Figure 6.3:** Finodil Item Listing use case diagram before and after revision

For RQ2, the result is that the modelers and validators reliably prevent the errors they encode, and the approval note makes the remaining flagged ones visible. They do not prevent errors of meaning. Names cut from sentences, generic messages and irrelevant participants pass, because they break no structural rule. The worst cases came from the fallback paths, where the model's contribution was smallest.

### 6.2.3 Completeness of the page designs (RQ3)

The UI/UX output for the retail store feature was considerably stronger. The metadata defines two pages, Add Item at /add-item and List Items at /list-items, each with one new component, AddItemForm and ItemListGrid, and an indigo colour theme. Between them the pages cover FR-001 and FR-002, both user stories and all five acceptance criteria. Figure 6.4 shows the two previews.

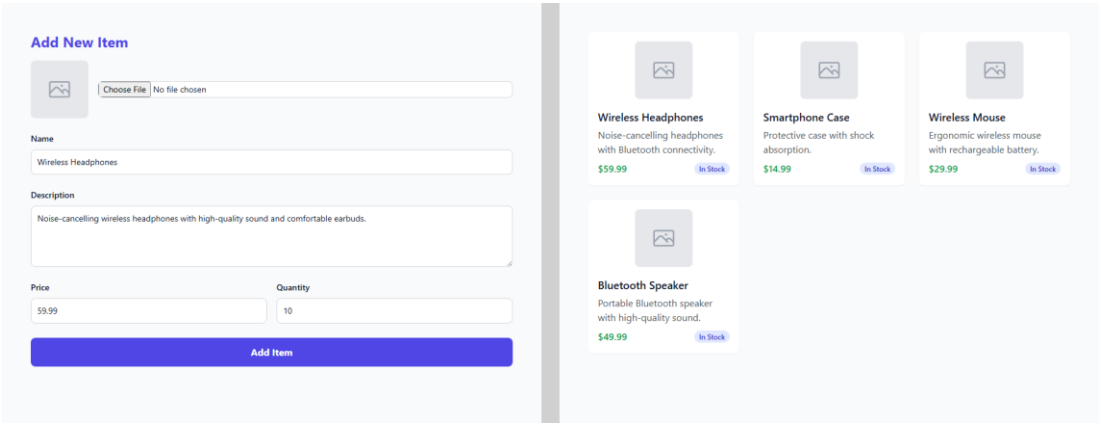



**Figure 6.4:** Retail store page previews: Add Item (left) and List Items (right)

Both pages show realistic, fully populated content: a form with the five fields from the SRS and example values, and a grid of four item cards with names, descriptions and prices. Neither contains placeholder text. Both components were later recorded in the project's design system as introduced by this feature, which only happens when the design is approved.

A closer comparison with the SRS and the architecture plan found four gaps, listed in Table 6.5.

**Table 6.5:** Gaps found in the retail store page design

| Gap | Why the checks did not catch it |
|---|---|
| No page for FR-DOM-001, viewing an item's details | The validator requires user stories and screen-related acceptance criteria, not functional requirements |
| The UI expectation "Responsive on both desktop and mobile widths" is not covered by any component | The SRS stores UI expectations as plain strings, but the validator only reads items with an "element" field, so the check never ran |
| Routes /add-item and /list-items differ from the plan's single /add-list-items page | No check compares the UI/UX metadata with the architecture plan's frontend pages |
| Item cards show an "In Stock" badge that no requirement asks for | Invented content passes the placeholder check by design |

The second gap is the most instructive. The coverage check was written and tested against UI expectations stored as objects, but the Requirement Agent's normalisation turns them into plain strings, so on real data the check silently did nothing. For RQ3, then: per-component generation, the placeholder gate and deterministic assembly

produced complete, realistic and reviewable pages, which is a clear improvement on the earlier React version that could show "Unknown state.". Coverage, however, was only as good as the rules, and one rule was not running.

### 6.2.4 Human review at the design stages (RQ4)

In the team's Item Listing case study, both of my stages needed revision before approval. The architecture plan and its diagrams were approved at version 3, after the reviewer corrected an actor in the diagrams. The UI/UX design was also approved at version 3, after component-level fixes. Across the whole pipeline, four of the seven stages of Item Listing were revised, and two of those four were mine. Table 6.6 shows the team's comparison of the full pipeline with human gates on and off.

**Table 6.6:** Pipeline with human gates on and off

| Measure | Gates ON | Gates OFF |
|---|---|---|
| Successful pipelines | 3/3 | 2/3 |
| Average revisions | 5.0 | 1.0 |
| QA pass rate | 91% | 78% |
| Security issues | 1 | 4 |
| Escaped defects | 1 | 5 |
| Human review effort | 36 min | 0 |

The development artefacts add detail to this. In the Finodil feature, the reviewer caught and removed a false actor that no validator had flagged. In the retail store feature, the approval note told the reviewer about the merged use case. Problems such as the broken data entity and the uncovered responsive expectation are exactly the kind a reviewer can see in seconds and a rule cannot. For RQ4, the evidence is that review at the design stages did catch real problems before code, and that it is needed most where the deterministic checks are weakest.

## 6.3 User or stakeholder validation

No external users or industry stakeholders evaluated the component. All reviews were done within the project team, whose members know the system and the expected output. A software architect or UI designer outside the team would probably judge the diagrams and pages more strictly, and that remains to be tested.

Feedback within the team did shape the design in at least two recorded cases. Human approval of the UI/UX stage was restored at the team's request after a period in which the stage was not gated, as the approval service's own notes record. And page-scoped revision, which at first was only a hint in the prompt, was made a hard rule after a revision meant for one page could also change others.

The panel gave overall positive feedback on the project. It did not comment specifically on the architecture plans, diagrams or page designs.

### 6.4 Integration results

My stages ran in every pipeline run of the team's evaluation, and all three features completed all seven stages with gates on. Table 6.7 shows the cost of each run. The logs do not separate the time spent in each stage, so I do not claim a figure for my stages alone.

**Table 6.7:** Execution performance of the full pipeline

| Run | Total time (min) | LLM calls | Retries | Tokens |
|---|---|---|---|---|
| Item Listing | 36.4 | 24 | 4 | 31,800 |
| Authentication | 52.7 | 31 | 6 | 42,600 |
| Shopping Cart | 56.8 | 34 | 7 | 47,900 |
| Baseline | 31.2 | 18 | 3 | 25,400 |

Two integration results concern my component directly. First, the Coder Agent consumed the implementation plan as intended: its planner prompt includes the plan's files, endpoints and models, as the integration tests confirm. Second, the integration problems found in Section 6.2 sit at the boundaries between agents: the data-requirement format between the Requirement and Architecture agents, the shape of UI expectations between the Requirement and UI/UX agents, and the page routes between the Architecture and UI/UX agents. None of them would have been found by testing one agent alone.

## 6.5 Summary of findings

**Table 6.8:** Summary of findings by research question

| RQ | Finding | Strength of evidence |
|---|---|---|
| RQ1 | Every required SRS identifier was traced, even on the fallback path, but a traced plan held a broken data entity | Validator tests; two stored plans |
| RQ2 | Modelers and validators prevent rule violations and surface some issues; errors of meaning, mostly from fallback paths, still pass | Tests; nine issues in five stored diagrams |
| RQ3 | Pages were complete, realistic and free of placeholders; one requirement and one expectation had no page, and routes disagreed with the plan | Tests; one feature's stored designs |
| RQ4 | Reviewers revised both design stages in the case study and caught issues no validator flagged | One case study; development revision records |

# 7 DISCUSSIONS

## 7.1 Interpretation of findings

The results support the idea behind the component, letting the model propose and letting code and a person decide, but they also show its limit more clearly than I expected. Deterministic checks are good at the question "is it all there?" and poor at the question "is it right?".

On the first question the component did well. No SRS identifier went missing from a plan, even when the model failed on every rung and the fallback took over. Every page design covered its user stories and screen-related acceptance criteria, and no preview showed placeholder content. These were real problems in early versions, and the deterministic steps removed them.

On the second question the results were mixed. The data entity in Table 6.3 is the clearest example. It satisfied every rule, yet a developer would reject it at a glance. The same is true of the generic sequence messages and the cut-off actor names. None of these break a structural rule, so a rule-based validator cannot see them. This is not an argument against validation; without it the plans and diagrams would be worse. It is an argument against reading a passing validator as approval.

A second pattern is that the weakest outputs came from the fallback paths. The fallback did its job, which was to give the reviewer something to look at instead of a failed stage, but its output is only as good as the parsers that turn SRS text into structure. When the SRS used a format the parser did not expect, the result was nonsense presented in a confident structure. The fallback therefore needs to say more loudly that it was used. At present the failure reason reaches the fallback builder but is not written into the plan, so only the template wording reveals what happened.

The UI/UX results were better than the architecture results, and I think the reason is the different attitude to failure. The UI/UX Agent has no fallback, generates small pieces one at a time, and assembles them in code. Each piece is small enough for the model to do well and for a simple check to judge. The Architecture Agent's

diagrams, by contrast, depend on the whole plan, and their deterministic defaults are generic by necessity.

### 7.2 Comparison with literature baselines and existing solutions

Like MetaGPT [2], my component turns a requirement document into a structured design and diagrams before any code is written. The differences are in checking and control. MetaGPT's design is reviewed by other agents; here, every design is checked against the SRS by code and then approved or revised by a person at a durable checkpoint, and the diagrams are checked against UML rules. My results suggest both matter. Code caught missing coverage that a person might overlook, and a person caught meaning errors that code could not see.

Compared with ChatDev [1], where design is part of an agent conversation, my component produces design artefacts that can be reviewed, versioned and approved separately. HULA [7] found that engineers value reviewing an agent's plan before coding. My case-study results are consistent with that: both design stages were revised before approval. Coding agents such as SWE-agent [3] and OpenHands [4] assume the design already exists; the implementation plan is my component's attempt to give a coding agent exactly that, in a form it can follow file by file.

On traceability, the plan's matrix provides the requirement-to-design link that Gotel and Finkelstein [5] describe, and it is checked on every plan. The results add a caution that the traceability literature [6] also implies: a link records that a relationship was claimed, not that it is correct.

### 7.3 Research contribution

The main research contribution of this study is showing how deterministic validation and human review can work together in the design stage of an LLM-based software development pipeline. Rather than depending only on the LLM to generate and check its own output, this work separates these responsibilities. The model generates the design, deterministic checks verify whether important requirements and structural rules are covered, and a human reviewer makes the final decision. The case-study results showed that this combination is useful because automated checks can identify

missing coverage, while human reviewers are still needed to notice design or meaning-related issues that pass those checks.

Another important contribution is the finding that traceability does not always mean correctness. The Architecture Agent can check whether every functional requirement, acceptance criterion, validation rule and non-functional requirement is represented in the architecture plan. However, the evaluation showed that a design can still be fully traced while containing poor or incorrect design decisions. For example, some generated data fields were technically present but were not meaningful for implementation. Similar issues were also found in UML diagrams. This shows that traceability is useful for checking completeness, but it should not be treated as proof that the design itself is correct.

The study also highlights an important issue in multi-agent systems: problems can happen because one agent produces information in a format that the next agent does not expect. Some of the weaknesses identified during evaluation came from these format mismatches rather than from the LLM itself. This suggests that multi-agent software development systems should pay more attention to the contracts and data formats shared between agents, not only to the quality of individual prompts or models.

Finally, the research was evaluated using 361 automated tests, a three-feature case study and stored artefacts from actual development runs. This helped identify both successful behaviours and real weaknesses that were not always visible through unit testing alone. Therefore, this work contributes practical evidence about the strengths and limitations of using rule-based validation together with LLM-generated software designs.

### 7.4 Product contribution

The main product contribution of this work is the development of the Architecture Agent and UI/UX Agent as part of the AutoForge pipeline. These two agents cover the design stage between the approved requirements and the Coder Agent. The Architecture Agent produces a structured architecture plan containing design views, traceability information and a file-level implementation plan that can later be used by

the Coder Agent. It also uses a four-stage generation approach so that the architecture stage can still produce a reviewable output even when the main LLM generation process fails.

Another contribution is the UML diagram generation process. The system produces use case, sequence and class diagrams without asking the LLM to directly write the final PlantUML code. Instead, the LLM first produces a smaller structured description of the diagram. Deterministic modelers and validators then apply UML rules, check the output and perform targeted repairs when necessary. This approach makes the diagrams easier to validate and more reliable for human review.

The UI/UX Agent also provides a practical way to generate and review interface designs before coding begins. It plans the required pages and components, checks whether they cover the approved requirements, generates individual HTML and Tailwind CSS components, and combines them into complete page previews. It also checks for placeholder content so that reviewers are not shown incomplete screens that only look finished. The approved UI/UX outputs are then made available to the Coder Agent as a visual and structural reference.

The product also includes support for revision, versioning and approval. Instead of regenerating the entire design when a reviewer requests a change, the system can apply targeted revisions to the affected parts. Architecture plans and page designs are stored as versioned artefacts, and downstream stages only continue after human approval. The shared design system is also updated only when a UI/UX design is approved.

Overall, the product contribution is not only the development of two AI agents. It is a complete design workflow that connects approved requirements with implementation through architecture plans, UML diagrams, page previews, validation, revision and human approval. This gives the Coder Agent a clearer and more structured input while also giving human reviewers the opportunity to detect problems before any code is generated.

## 7.5 Limitations and threats to validity

### 7.5.1 Threats to Validity

Internal validity. The artefact analysis was done by a member of the team that built the agents. I tried to reduce bias by tying every finding to a stored file and reporting weak outputs, but another reviewer might judge some diagrams differently.

External validity. The detailed analysis covers two development features, and the case study three features, all in e-commerce and all targeting one stack. Plans for larger features, or for a different stack, may behave differently. Most results come from one default model; a stronger model would probably reach the fallback paths less often.

Construct validity. Counting traced identifiers measures coverage, not design quality, which is precisely the point made in Section 6.2.1. Diagram quality was judged by inspection against UML conventions rather than with a formal metric.

### 7.5.2 Limitations

- The plan validator checks that a data entity exists, not that its fields are sensible, and the fallback's field parser recognizes only the "name (details)" form.
- The fallback plan does not record that the model failed. The failure reason is passed to the fallback builder but never written into the plan, so only the template wording gives it away.
- The UI/UX coverage check for UI expectations reads only object-shaped items, so it does not run on the plain-string expectations the Requirement Agent actually produces.
- The UI/UX validator does not require every functional requirement to have a page, and does not compare its routes with the architecture plan.
- The diagram validators cannot detect meaning errors such as names cut from sentences, duplicate actors with plural names, or irrelevant participants.
- No architect or designer outside the team reviewed the outputs.

### 7.6 Implications

Three lessons may be useful to anyone building a similar stage. First, test deterministic checks against the real output of the previous agent, not only against hand-written fixtures. Two of the most important problems I found, the data-requirement format and the plain-string UI expectations, passed every unit test because the fixtures used the shape the code expected. Second, make a fallback announce itself. A safety net that produces plausible-looking output can do more harm than an honest failure if the reviewer does not know it was used. Third, give the reviewer the right view: diagrams and live page previews are much faster to judge than JSON, and most of the meaning errors in Chapter 6 are obvious once seen.

# 8 COMMERCIALISATION AND SUSTAINABILITY

## 8.1 Value of the Component

The team's common report sets out the commercial case for AutoForge as a whole: teams that build software with AI need to see and approve each stage before it ships. My component contributes to the stage where most of the visible design work happens, and the stage a paying client is most likely to look at. Small agencies often skip written design for small features because it takes time, and clients often ask for changes only after a screen has been built. Both habits make rework expensive. AI tools have not removed this problem: in the 2025 Stack Overflow Developer Survey, 66% of developers said their biggest frustration with AI was output that is "almost right, but not quite" [20]. A design that is almost right is much cheaper to correct as a diagram or a preview than as finished code.

My component offers two outputs that would be useful even outside the full pipeline. The first is a design pack for each feature: an architecture plan organised into views, a traceability matrix, three UML diagrams and a file-level implementation plan for the Coder Agent. Every identifier the validator requires was traced, even on the fallback path, with 36 rows in the retail store matrix (Table 6.2). The second is a set of page previews with realistic content that a client can approve before any code is written. In the retail store feature both pages were complete, realistic and free of placeholder text (Section 6.2.3). The previews open offline and every link in them is made inert, so an agency can safely send them to a client. Review of these outputs was not a formality: both of my stages needed revision before approval in the Item Listing case study, and the reviewer removed a false actor that no validator had flagged (Section 6.2.4).Table 8.1 shows who would use the component and what each group gains.

**Table 8.1:** Target users of the component

| User group | Need | What the component provides |
|---|---|---|
| Developers and technical leads in small agencies | A design for each feature without spending a day on it | A traced architecture plan, UML diagrams and a file-level implementation plan as a first draft to correct |
| Designers | Screens that match the requirements and stay consistent across features | Page previews built component by component, and a shared design system updated only on approval |
| Clients of those agencies | See and approve the screens before paying for development | Offline page previews with realistic content that can be shared safely |
| Lecturers and students | Teach architecture and UML with real examples | Plans and diagrams traced to a specification, with the UML rules each one was checked against |

This evidence comes from our own study, and no architect or designer outside the team has yet evaluated the component (Section 6.3). The team is therefore surveying developers, designers and agency staff, as described in Section 7.1 of the common report.

### 8.2 Target Users and Market Position

The first market is the one AutoForge targets: small e-commerce development teams and agencies, beginning in Sri Lanka, whose IT and business process management industry has more than 680 companies and over 700 active startups [21]. Agencies that build stores for clients are the users who gain most from this component, because client sign-off on design is part of their everyday work.

Table 8.2 compares the component with what these users can do today. The prices are published plan prices in October 2026, converted at 330.43 rupees to the dollar [27].

**Table 8.2:** Alternatives for producing a feature design

| **Alternative** | **Typical cost** | **Strength** | **Weakness compared with this component** |
|---|---|---|---|
| Designing by hand | Developer or designer time | Full judgement and context | Slow; often skipped for small features; traceability rarely kept |
| General chat assistant (ChatGPT Business, Claude Team) | USD 25 (about LKR 8,260) per user per month [25], [26] | Fast and flexible | No fixed structure, coverage check or UML rules; output not tied to requirement IDs |
| Prompt-to-UI and app builders (Lovable, Bolt) | USD 25 to 50 per user per month [23], [24] | Attractive screens quickly | Not tied to a specification or an architecture; no coverage check; no design record |
| Role-based frameworks such as MetaGPT [2] | Free (open source) | Design documents within a pipeline | Reviewed by agents, not by a person at a durable gate |

| Alternative | Typical cost | Strength | Weakness compared with this component |
|---|---|---|---|
| This component (within AutoForge) | Included from the free Community edition | Traced plan, UML diagrams, page previews, human approval, shared design system | Slower than a single prompt; output still needs expert review |

The component's competitive position rests on the link between design and requirements. Prompt-to-UI tools produce attractive screens, but nobody can say which requirement a screen serves or which one it forgets. My component ties every plan item and every page to the identifiers of the approved specification and checks the diagrams against UML rules before a person sees them. The shared design system adds a second advantage that grows with use: the more features an agency builds for a client, the more approved components are reused word for word.

### 8.3 Business Model and Feasibility

The component fits within the open-core model and the packages proposed in the common report (Table 7.5): a free self-hosted Community edition, Team seats at LKR 6,000 per month, Agency seats at LKR 10,000 per month, and enterprise and academic licenses. The Agency package in particular is built mostly on this component. Table 8.3 shows the parts of the model that depend directly on my work.

**Table 8.3:** Revenue elements that depend on this component

| Revenue element | Price (common report) | What this component contributes |
|---|---|---|

| Revenue element | Price (common report) | What this component contributes |
|---|---|---|
| Agency package: a design system per client | LKR 10,000 per seat per month | The design system is already kept per project and changes only on approval, so one project per client gives each client its own look |
| Agency package: client sign-off of previews | Included in Agency | Offline, link-safe page previews; page-scoped revision and version history already exist |
| Team review features | LKR 6,000 per seat per month | Approval cascades, revision and version history for plans, diagrams and pages |
| Setup services | LKR 75,000 per engagement | Building a client's existing brand into the design system before the first feature |
| Academic license | LKR 150,000 per year per campus | Traced plans and rule-checked UML diagrams for teaching software design |

The value to a buyer can be estimated roughly. If drafting the plan, three diagrams and two-page mock-ups for a feature would take a developer half a day to a day by hand, that is about LKR 4,000 to 16,000 at typical Sri Lankan software engineering salaries. The component produces a first draft within one pipeline run, which costs nothing in API fees with the local model and at most about LKR 160 on a hosted model. Correcting a draft is faster than starting from nothing, so the Agency seat price is recovered after a few features a month. The half-day-to-a-day figure is an assumption, and the team's pilot is designed to measure it.

Technically, the component is feasible on a single machine with a local model, as the evaluation shows. The results in Chapter 6 also point to the work needed before a

commercial release, listed in Table 8.4. A product sold on trust cannot ship with a check that silently does not run.

**Table 8.4:** Work needed before a commercial release

| Issue found | Evidence | Change needed |
|---|---|---|
| The fallback plan looked complete but held a broken data entity | Tables 6.2 and 6.3 | The fallback announces itself to the reviewer, and data-model checks look at field names and types |
| A coverage check never ran on real data | Table 6.5 | Test every check against real output from the Requirement Agent, not only fixtures |
| Page routes disagreed with the architecture plan | Table 6.5 | A check that compares the UI/UX metadata with the plan's frontend pages |
| Errors of meaning in diagrams passed the validators | Table 6.4 | Present checks as help for the reviewer; route fallback diagrams to closer review |
| No architect or designer outside the team has reviewed the outputs | Section 6.3 | A pilot with one agency, measuring how often plans, diagrams and pages are accepted, corrected or rejected |

### 8.4 Sustainability and Maintenance

Several design choices help the component stay maintainable. PlantUML and Tailwind are text formats that change slowly and can be stored in version control. The validators and modelers contain no feature-specific rules, so new features do not require new code. The implementation plan is tied to the Coder Agent's file conventions, so if the generated stack changes, the plan builder and its tests must change with it; keeping both in one place makes that change visible.

The main maintenance risks are the dependencies on PlantUML with Java and Graphviz, and on Playwright with Chromium, which add installation steps, and the growing size of the Architecture Agent's main module. Splitting the fallback plan builder and the diagram tiers into their own modules would make the agent easier to change.

Socially, the component keeps designers and developers in charge. It produces a first draft and a set of checks; a person decides. The risk discussed in Chapter 7 applies here too: a design that passes every check can still be wrong, so the product should present checks as help for the reviewer, not as a guarantee.

### 8.5 Contribution to the Sustainable Development Goals

The common report links AutoForge to SDG 9 (Industry, Innovation and Infrastructure) and SDG 8 (Decent Work and Economic Growth). My component's contribution to SDG 9, and in particular target 9.5 on upgrading technological capabilities, is a way to bring architecture and interface design, which small teams often skip, into their everyday process at low cost, using a model that can run locally. Its contribution to SDG 8, targets 8.2 and 8.3, would come through small agencies, which could show clients screens and designs earlier and spend less time on rework.

These contributions are potential rather than demonstrated. There are also possible negative effects. A design that passes every check can still be wrong, and reviewers may come to trust the checks too much; junior developers may also practise design less if a first draft always appears. The component should therefore present its checks as help for the reviewer rather than a guarantee, and keep the reviewer's corrections visible in the version history, where they also serve as a way to learn. A responsible next step would be a pilot with a small agency, measuring how often their developers accept, correct or reject the generated plans, diagrams and pages, and how much review time the stage saves or costs.

# 9 CONCLUSION AND FUTURE DIRECTIONS

## 9.1 Conclusion

This thesis set out to make the design stages of an LLM-based development pipeline produce designs that are traceable to the approved specification, follow modelling and interface conventions, and are reviewed by a person before code is written. The Architecture Agent produces a structured plan through a four-rung ladder, checks its coverage of the SRS, and generates UML diagrams through narrow specifications, deterministic modelers and rule validators. The UI/UX Agent plans pages and components, checks their coverage, generates each component behind a placeholder check, and assembles self-contained previews, with the shared design system updated only on approval.

The evaluation showed that the deterministic steps reliably guarantee completeness: no requirement went untraced, and no preview showed placeholder content. It also showed that completeness is not correctness. A fully traced plan carried a broken data entity, diagrams passed every rule while containing names cut from sentences and irrelevant participants, and a coverage check never ran because two agents disagreed on the shape of the data. In the case study, reviewers revised both design stages before approving them. The main lesson of this work is that deterministic checks and human review do different jobs, and a design stage needs both.

## 9.2 Achievement of objectives

**Table 9.1:** Achievement of the objectives

| Objective | Outcome | Status |
|---|---|---|
| O1: Architecture plan with implementation plan and SRS coverage check | Fifteen-section plan, eight views, implementation plan, four-rung ladder, coverage validator; data-model content not checked | Achieved |
| O2: UML diagrams through specifications, modelers and validators | Three diagram types with modelers, validators, targeted repair and PlantUML rendering; meaning errors still pass | Largely achieved |

| Objective | Outcome | Status |
|---|---|---|
| O3: UI/UX pages with coverage, realistic components, previews and shared design system | Implemented with coverage repair, content gate, reuse, page assembly, previews and approval-gated design system; one coverage check not running on real data | Largely achieved |
| O4: Evaluation including where checks fall short | 361 automated tests, case-study records and analysis of stored artefacts from two features | Achieved at limited scale |

### 9.3 Individual research and product contributions

The main contribution is a tested design for the architecture and UI/UX stages of a human-gated agent pipeline: a traceable architecture plan with a file-level implementation plan for a coding agent, a diagram pipeline that keeps the model away from the notation and checks the result against UML rules, and a page design pipeline that generates small pieces, rejects placeholders and assembles previews in code. The second contribution is evidence of where such checks stop working, drawn from real artefacts rather than test fixtures: format mismatches between agents, checks that silently do not run, and fallback output that looks complete. These failure modes are likely to appear in any multi-agent pipeline, and they are what a reviewer of such a system should look for.

### 9.4 Future directions

The most direct improvements follow from Chapter 6. The fallback plan should state in its approval note that the model failed. The data-entity check should look at field names and types, for example rejecting fields named after types such as "string" or "number", and the field parser should accept the "name: type" form. The UI/UX coverage check should accept UI expectations as plain strings, require a page for every functional requirement with a visible effect, and compare its routes with the architecture plan's front-end pages.

Beyond these fixes, contract tests between agents would catch format mismatches early: each agent's checks should be run against the real stored output of the agent before it, not only against fixtures. Diagram quality could be improved by checking names against the vocabulary of the SRS, so that a name cut from a sentence or an

actor not mentioned in the SRS is flagged. Finally, the evaluation should be repeated with more features, more than one model, and reviewers from outside the team, ideally an architect for the plans and a designer for the pages.

# REFERENCES


[1] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 15174–15186.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 23247–23275.

[3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 50528–50652.

[4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025, pp. 65882–65919.

[5] O. C. Z. Gotel and A. C. W. Finkelstein, "An analysis of the requirements traceability problem," in *Proc. IEEE Int. Conf. Requirements Engineering*, 1994, pp. 94–101.

[6] J. Mucha, A. Kaufmann, and D. Riehle, "A systematic literature review of pre-requirements specification traceability," *Requirements Engineering*, vol. 29, no. 2, pp. 119–141, 2024.

[7] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2025, pp. 342–352.

[8] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.

[9] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, 2025.

[10] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, 2024.

[11] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, “When software security meets large language models: A survey,” *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, 2025.

[12] K. Pelluru, “LangChain & LangGraph in production: Architectures for multi-agent LLM systems,” *Journal of Data and Digital Innovation (JDDI)*, vol. 2, no. 3, pp. 1–9, 2025.

[13] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[14] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” *arXiv preprint* arXiv:2402.01030, 2024.

[15] H. N. Phan, T. N. Nguyen, P. X. Nguyen, and N. D. Bui, “HyperAgent: Generalist software engineering agents to solve coding tasks at scale,” *arXiv preprint* arXiv:2409.16299, 2024.

[16] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based software engineering agents,” *arXiv preprint* arXiv:2407.01489, 2024.

[17] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024, pp. 54107–54157.

[18] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, “AgentCoder: Multi-agent-based code generation with iterative testing and optimisation,” *arXiv preprint* arXiv:2312.13010, 2023.

[19] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, “RepoCoder: Repository-level code completion through iterative retrieval and generation,” in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 2471–2484.

[20] Stack Overflow, “2025 Developer Survey: AI,” 2025. [Online]. Available: https://survey.stackoverflow.co/2025/ai (accessed Oct. 8, 2026).

[21] “SLASSCOM reflects on a year of creating opportunities, strengthening Sri Lanka’s knowledge and innovation industry,” Daily FT, Jul. 15, 2026. [Online]. Available: https://www.ft.lk/it-telecom-tech/SLASSCOM-reflects-on-a-year-of-creating-opportunities-strengthening-Sri-Lanka-s-knowledge-and-innovation-industry/50-794672 (accessed Oct. 8, 2026).

[22] “Data protection compliance regime takes effect on 1 Jan. 2027,” Daily FT, 2026. [Online]. Available: https://www.ft.lk/front-page/Data-protection-

compliance-regime-takes-effect-on-1-Jan-2027/44-795776 (accessed Oct. 8, 2026).

[23] Lovable, “Subscription plans.” [Online]. Available: https://docs.lovable.dev/introduction/subscription-plans (accessed Oct. 8, 2026).

[24] StackBlitz, “Bolt pricing.” [Online]. Available: https://bolt.new/pricing (accessed Oct. 8, 2026).

[25] “ChatGPT pricing in 2026: Free, Go, Plus, Pro, Business and Enterprise,” GPTPrompts.ai. [Online]. Available: https://gptprompts.ai/ai-pricing/chatgpt-pricing (accessed Oct. 8, 2026).

[26] Anthropic, “Claude pricing.” [Online]. Available: https://claude.com/pricing (accessed Oct. 8, 2026).

[27] Exchange-Rates.org, “USD to LKR: Convert US Dollars to Sri Lanka Rupees.” [Online]. Available: https://www.exchange-rates.org/converter/usd-lkr (accessed Oct. 7, 2026).

# GLOSSARY

| | |
|---|---|
| **Actor** | In a use case diagram, a person or external system that interacts with the system. |
| **Approval cascade** | Applying one approval decision to every artefact of the same stage version. |
| **Architecture plan** | The structured design document the Architecture Agent produces for one feature. |
| **Design system** | A project's shared record of interface components and design tokens, reused across features. |
| **Design view** | One section of the plan describing the design from a single perspective, such as data or behaviour. |
| **Fallback** | A deterministic path that produces output without the model when the model's output cannot be used. |
| **Implementation plan** | The part of the plan listing the files, endpoints, models and build order for the Coder Agent. |
| **Integration manifest** | A JSON description of each page's route, navigation entry and components for the Coder Agent. |
| **Modeler** | Code that turns a diagram specification into a final UML model using fixed rules. |
| **PlantUML** | A text language for UML diagrams that is rendered into images. |
| **Tailwind CSS** | A CSS framework of small utility classes applied directly in HTML. |
| **UML** | Unified Modeling Language, a standard notation for describing software structure and behaviour. |
| **Use case** | A goal an actor achieves with the system, shown as an oval in a use case diagram. |

# APPENDICES

## Appendix A Supporting Technical Material

### A.1 Test catalogue

**Table A.1:** Test files for the component

| Test file | Functions | What it covers |
|---|---|---|
| test_architecture_agent_exploration | 7 | Agentic rung, tools, step limit, fallback |
| test_architecture_agent_revision_ladder | 4 | Revision ladder |
| test_architecture_agent_transport_errors | 4 | Timeouts and failed calls |
| test_architecture_data_requirements_consolidation | 15 | Field and endpoint parsing, one entity |
| test_architecture_enhanced_srs | 5 | Enhanced SRS preference; Coder planner prompt |
| test_architecture_pdf_builder | 10 | Plan PDF |
| test_architecture_plan_schema | 8 | Plan schema |
| test_architecture_project_context | 5 | Earlier plans and manifest as context |
| test_architecture_reviewer_note_builder | 6 | Plain-English approval note |
| test_architecture_revision_patcher | 23 | Targeted plan operations |
| test_architecture_validation_error_handling_dedup | 4 | Error-handling view wording |
| test_architecture_usecase_modeler | 25 | Use case normalisation |
| test_architecture_usecase_validator (3 files) | 24 | Structure, name quality, traceability |
| test_architecture_usecase_builder, _repair | 11 | PlantUML output; targeted repair |
| test_architecture_usecase_diagram_exploration, _tools | 19 | Agentic use case loop and tools |
| test_architecture_sequence_modeler | 14 | Sequence model |
| test_architecture_sequence_validator (2 files) | 11 | Fragments, messages, quality |
| test_architecture_sequence_builder, _repair | 8 | PlantUML output; targeted repair |
| test_architecture_class_modeler, _repair | 11 | Class model; targeted repair |

| Test file | Functions | What it covers |
|---|---|---|
| test_architecture_class_validator_quality | 16 | Class rules |
| test_architecture_diagram_exploration, _tools | 26 | Sequence and class loops and tools |
| test_approval_architecture_cascade | 6 | Plan approval cascade |
| test_uiux_metadata_modeler, _validator | 18 | Metadata and coverage |
| test_uiux_component_generator, _agent_quality_gate | 13 | Component generation and content gate |
| test_uiux_component_reuse | 4 | Approved component reuse |
| test_uiux_page_html_builder | 7 | Page assembly |
| test_uiux_integration_manifest_builder | 3 | Manifest |
| test_uiux_revision_patcher, _agent_revision | 42 | Targeted component operations; revision flow |
| test_uiux_images_download | 5 | Zip download of previews |
| test_approval_uiux_cascade | 7 | UI/UX approval cascade and design system |
| **Total** | **361** | |

### A.2 Retail store use case diagram source, version 1

The PlantUML source below is the one the Architecture Agent saved, with only its comment and style lines removed. It contains the merged use case and the truncated external actor discussed in Section 6.2.2.

```
@startuml
    left to right direction
    title Add & List Items Use Case Diagram

    actor "Any User" as ACT_001
    actor "Image Hosting Solution To" as ACT_002 <<external system>>

    rectangle "Add & List Items Feature" {
      usecase "Add List Items" as UC_001
    }

    ACT_001 -- UC_001
    ACT_002 -- UC_001
    @enduml
```

## Appendix B: Disclosure of AI Tool Use

**Table B.1:** Use of generative AI tools

| Activity | Tool | How it was used | How the output was checked |
|---|---|---|---|
| System runtime | qwen3-coder via Ollama | The model inside the Architecture and UI/UX agents, as described in this thesis | Deterministic validation, tests and human review |
| Figures | Claude (Anthropic) | Drawing the design diagrams. | Checked against the implementation by the author |